



Draft Less, Retrieve More: Hybrid Tree Construction for Speculative Decoding

Yuhao Shen^{1,2*}, Tianyu Liu^{2,3*}, Xinyi Hu^{2,1*}, Quan Kong¹, Baolin Zhang^{2,1}, Jun Dai², Jun Zhang^{2,1†}, Shuang Ge², Lei Chen², Yue Li², Mingcheng Wan² and Cong Wang^{1†}

¹Zhejiang University, ²Qwen Applications Business Group of Alibaba, ³University of Science and Technology of China

*Core Contribution. †Corresponding author.

Speculative decoding (SD) accelerates large language model inference by leveraging a *draft-then-verify* paradigm. To maximize the acceptance rate, recent methods construct expansive draft trees, which unfortunately incur severe VRAM bandwidth and computational overheads that bottleneck end-to-end speedups. While dynamic-depth pruning can reduce this latency by removing marginal branches, it also discards potentially valid candidates, preventing the acceptance rate from reaching the upper bound of dense trees. In this paper, we identify a critical opportunity in resource allocation: the transition from dense to pruned drafting frees up significant computational budget. To break this Pareto tradeoff, we introduce Graft, a compensation framework that **couples pruning and retrieval as mutually reinforcing operations**. Pruning supplies sufficient budget for retrieval, while retrieval compensates for pruning-induced coverage loss and recovers accepted length. By employing a sequential ‘prune-then-graft’ mechanism, Graft attaches highly predictive retrieved tokens into positions opened by pruning, filling the topological gaps with near-zero overhead. Graft is entirely training-free and lossless. Comprehensive evaluations show that Graft establishes a **new Pareto frontier** across practical deployment settings, including short-context generation, long-context generation, and large-scale models. On short-context benchmarks, it achieves up to **5.41×** speedup and improves average speedup over EAGLE-3 by up to **21.8%** on the large-scale Qwen3-235B. On long-context benchmarks, Graft reaches **3.22×** average decoding speedup on LLaMA3.1-8B and outperforms EAGLE3-64K by **16.6%** on Qwen3-14B. We also provide a preliminary exploration of applying Graft to the DFlash-style block drafting paradigm, offering initial evidence and insights for extending grafting beyond autoregressive draft trees.

1. Introduction

Autoregressive decoding in large language models (LLMs) is inherently sequential: every generated token depends on the prefix produced so far (Brown et al., 2020). As LLMs scale toward larger parameter counts and longer context windows (Yang et al., 2025a; Guo et al., 2025), this sequential dependency becomes a persistent latency bottleneck. System-level optimizations, such as quantization, distillation, and efficient attention (Hinton et al., 2015; Dao et al., 2022; Choi et al., 2018), reduce the cost of each forward pass. However, they do not alter the *token-by-token* nature of generation. This problem is especially pronounced in real-world deployments, where long outputs and extensive KV-cache overhead amplify even minor inefficiencies in the decoding loop.

Speculative decoding (SD) losslessly relieves this bottleneck via a *draft-then-verify* paradigm (Leviathan et al., 2023; Chen et al., 2023; Zhang et al., 2024a). Traditionally, a lightweight draft model proposes a sequential chain of candidate tokens, which the target model then verifies in a single parallel forward pass. To further increase the mean accepted length (MAT) per step, recent methods evolve this chain into a token tree, verifying multiple candidate branches simultaneously (Miao et al., 2024a). Notably, EAGLE-3 (Li et al., 2025) serves as a strong practical baseline. It reuses target-model features to predict subsequent tokens, thereby constructing high-quality draft trees. However, this tree-based expansion exposes a significant challenge. While incorporating more candidate branches provides broader coverage and higher MAT, it also heavily increases

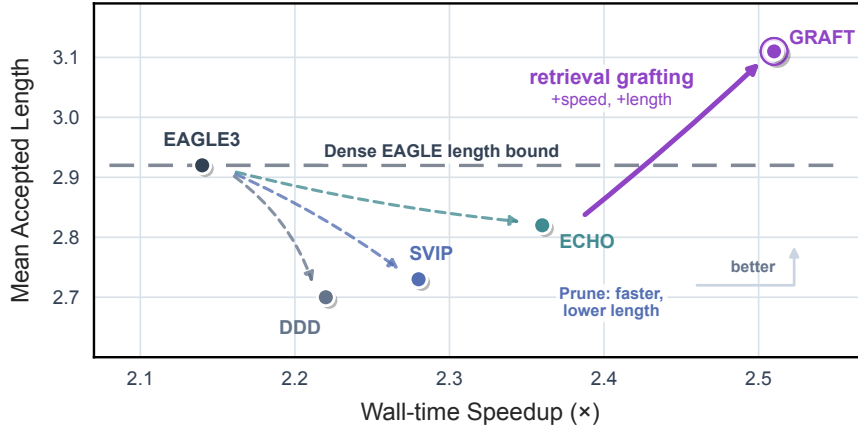


Figure 1 | **Speed-accepted-length tradeoff on Qwen3-32B HumanEval.** Each point reports wall-time speedup and mean accepted length. Dense EAGLE3 gives the accepted-length upper point for pruning-only subtrees. Dynamic pruning methods such as DDD, SVIP, and ECHO move rightward by reducing draft cost, but their accepted length falls below the dense-tree bound. Graft uses retrieval to fill the slots released by pruning, introducing candidates beyond the original subtree and breaking this pruning trade-off under the same verification budget.

draft-side search, memory bandwidth consumption, and verification workload. Consequently, the increased acceptance rate brought by a large draft tree often fails to translate into optimal end-to-end wall-clock speedups.

Dynamic tree construction addresses this inefficiency through dynamic-depth pruning (Brown et al., 2024; Hu et al., 2026; Liu et al., 2026a; Zhang et al., 2024c). When the draft model is uncertain, the tree is pruned at a shallower depth to avoid wasted computation. Yet, this introduces a structural limitation: dynamic trees are strictly subtrees of the original static tree. Because confidence signals are noisy, fine-grained pruning inevitably causes misjudgments. The controller may prune away valid continuations, strictly bounding the MAT below the static-tree upper limit. **This creates a strict latency-MAT frontier**, as vividly illustrated in Figure 1. Dynamic pruning methods such as DDD, SVIP, and ECHO successfully run faster by reducing draft cost (moving rightward), but their accepted length inevitably falls below the dense EAGLE-3 bound. While such pruning errors are unavoidable, we argue that they can be mitigated by effectively repurposing the computational redundancy created by pruning.

Our key observation is that **pruning acts not merely as candidate removal but as a critical mechanism for budget release**. Once low-confidence branches are pruned, the freed slots need not remain empty, nor should they be wasted on deeper uncertain drafting. Instead, they can be filled by a cheaper candidate source: retrieval. Prior retrieval-based SD methods have explored prompt lookup, external datastores, and cached transitions (Saxena, 2023; He et al., 2024; Luo et al., 2024; Fu et al., 2024). However, most act as standalone drafters or target-side auxiliary mechanisms. For instance, SAM-Decoding (Hu et al., 2025) only uses retrieval quality as a routing signal to decide whether to invoke a parametric tree drafter, rather than enriching the draft tree itself. Furthermore, many existing retrieval methods rely on CPU-side lookup or synchronization-heavy structures, creating overheads that easily negate speculation gains. By seamlessly integrating retrieval to fill the topological gaps left by pruning, we can compensate for misjudgments and create a new performance bound.

Motivated by this observation, we introduce Graft, a training-free and lossless hybrid tree construction framework built around a **prune-then-graft** strategy under the principle of **draft less, retrieve more**. Graft first performs confidence-based pruning through calibrated pruning checkpoints: when draft confidence is low, it prunes the tree and releases the remaining candidate budget. It then grafts retrieval-based branches into the released slots, keeping the final verification budget **identical to the original static tree**. As shown in Figure 1, this allows Graft to introduce candidates beyond the original subtree, effectively **breaking the pruning trade-off**. The two steps are complementary by construction: **pruning supplies budget for retrieval**, while **retrieval compensates for pruning-induced coverage loss** with context-aware continuations. To ensure production viability, the retrieval branch is rooted at the current token and prepared in parallel with autoregressive drafting. Graft utilizes a GPU-resident adjacency matrix, initializes it via warm-up, and updates it online using target-model verification signals. This design preserves the standard tree-attention path, avoids extra target forward passes, and removes CPU-side synchronization. Crucially, this architecture naturally scales

to long-context generation. As the prompt extends, autoregressive drafting becomes increasingly expensive, making pruning more rewarding. Simultaneously, the extended context provides richer local transition patterns, which naturally boosts the retrieval hit rate without additional overhead.

In summary, this paper makes the following contributions:

- (1) **A budget-compensation view of dynamic tree pruning.** We analyze the **latency-MAT frontier** of dynamic tree construction. We show that pruning-only methods, restricted to static subtrees, inevitably lose MAT. This motivates treating pruned slots as reusable computational budget rather than discarded candidates, establishing the foundation for effective retrieval integration.
- (2) **Graft: GPU-friendly prune-then-graft construction.** We propose grafting retrieved candidates into slots released by pruning. **Pruning supplies budget for retrieval**, while **retrieval compensates for pruning-induced candidate loss**. Utilizing root-centered parallel retrieval, a GPU-resident adjacency matrix, and online target-guided updates, Graft packs candidates into the standard verification path. This expands the candidate set beyond the original subtree while preserving the target verification budget and ensuring lossless decoding.
- (3) **Scaling to deployment settings.** Experiments show that Graft establishes a **new speed-MAT frontier** across short-context, long-context, and large-scale deployments. It achieves up to **5.41×** speedup on short-context tasks and improves average speedup over EAGLE-3 by up to **21.8%** on the **large-scale Qwen3-235B**. For long-context LLaMA3.1-8B, it reaches **3.22×** average speedup, outperforming EAGLE3-64K by **16.6%** on Qwen3-14B.

As a bonus, we also provide a preliminary exploration of applying Graft to the DFlash-style block drafting paradigm, offering initial evidence and insights for extending grafting beyond autoregressive draft trees.

2. Preliminary Study

2.1. The Dynamic-Tree Frontier

Standard SD and the dynamic-tree bound. Speculative decoding (SD) accelerates autoregressive generation through a draft-then-verify pipeline (Leviathan et al., 2023; Chen et al., 2023). At each step, the draft model proposes a token tree $\mathcal{T}$ for the current prefix $x_{1:t}$, which the target model verifies in a single parallel forward pass (Miao et al., 2024b). The generation progress per step is measured by the mean accepted length (MAT), $M(\mathcal{T}) = \mathbb{E}[L(\mathcal{T})]$, while the computational cost is $C(\mathcal{T}) = T_{\text{draft}}(\mathcal{T}) + T_{\text{verify}}(\mathcal{T})$. A standard proxy for wall-clock speedup is:

$$S(\mathcal{T}) = \frac{(M(\mathcal{T}) + 1)T_{\text{ar}}}{C(\mathcal{T})}. \quad (1)$$

EAGLE-3 employs a dense tree $\mathcal{T}_E$ to maximize $M(\mathcal{T})$ through broader candidate coverage (Li et al., 2025). However, this dense structure substantially increases draft-side search, memory bandwidth, and verification workload. Dynamic-tree methods (e.g., DDD, SVIP, ECHO) mitigate this overhead through dynamic-depth pruning (Brown et al., 2024; Hu et al., 2026). While this reduces the cost denominator in Eq. 1, it introduces a strict structural ceiling. The pruned trees remain constrained as subtrees of the original dense tree:

$$\mathcal{T}_\pi \subseteq \mathcal{T}_E \implies M(\mathcal{T}_\pi) \leq M(\mathcal{T}_E). \quad (2)$$

This bound illustrates the core limitation of pruning-only designs. Removing nodes accelerates drafting but cannot introduce novel candidate paths. As a result, MAT is strictly capped by the dense-tree upper bound.

Latency versus MAT. Dynamic-depth pruning imposes a strict trade-off rather than a pure acceleration. Relative to the dense tree, a dynamic policy π modifies the speedup ratio as follows:

$$\frac{S(\mathcal{T}_\pi)}{S(\mathcal{T}_E)} = \underbrace{\frac{M(\mathcal{T}_\pi) + 1}{M(\mathcal{T}_E) + 1}}_{\text{MAT loss}} \cdot \underbrace{\frac{C(\mathcal{T}_E)}{C(\mathcal{T}_\pi)}}_{\text{latency saving}}. \quad (3)$$

The latency saving improves as pruning eliminates expensive draft computation. However, the MAT loss worsens when the controller over-prunes valid continuations. Since confidence is merely a proxy for target acceptance,

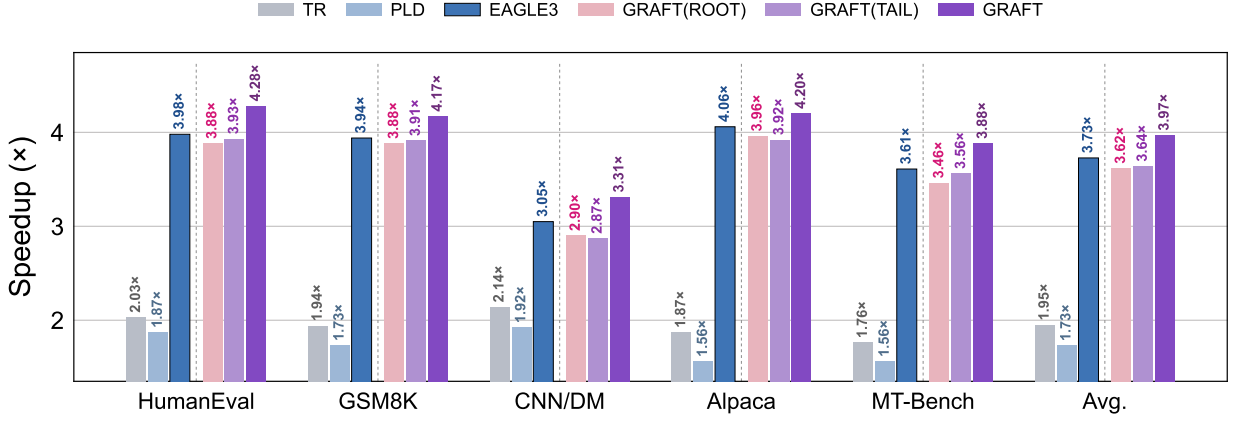


Figure 2 | **Motivation of Graft.** Speedup comparison on LLaMA3.1-8B across five benchmarks and their average. Traditional self-SD retrieval methods such as PLD and TR are lightweight but remain far below EAGLE3. Under the same total candidate budget, direct grafting variants are also limited. **Graft**(ROOT) competes with original candidates, while **Graft**(TAIL) depends on a fully accepted draft prefix. **Graft** couples pruning and retrieval. Pruning releases enough budget for retrieval, and retrieval repairs the coverage loss caused by pruning, achieving the best average speedup.

this risk is unavoidable. With dense depth-wise gating, over-pruning compounds. If ϵ_d is the probability of pruning a useful continuation at depth d , the chance of at least one harmful pruning decision along a path grows as $1 - \prod_d (1 - \epsilon_d)$. Dynamic trees therefore operate along a strict latency-MAT frontier. They save time by drafting less but pay the price in reduced accepted tokens.

This frontier highlights that pruning acts fundamentally as a mechanism for *budget release* rather than mere candidate removal. Once low-confidence branches are pruned, the freed candidate slots need not remain empty. Instead of reinvesting these slots into highly uncertain autoregressive drafting, we can ask: can this released budget be repopulated by a cheaper candidate source unconstrained by the original subtree?

2.2. Retrieval as Budget Compensation

This budget-release perspective points to non-parametric retrieval. Local continuations from the prompt or generation history can be reused with minimal additional computation. However, retrieval alone cannot replace the original drafter. Systems like PLD and Token Recycling (TR) reuse past transitions to accelerate the target model directly (Saxena, 2023; Luo et al., 2024). They operate in isolation and fail to combine retrieved candidates with generated draft tokens. Without this synergy, their speedup ceilings are severely constrained and fall far below strong tree-based baselines. Many retrieval systems also rely on CPU-side datastores or synchronization-heavy logic. This creates overhead that easily negates speculation gains in large-model serving. Retrieval is an attractive compensation source but remains insufficient on its own.

How should retrieval be incorporated into a fixed-budget draft tree? Two straightforward insertion strategies highlight the critical importance of budget allocation. **Graft**(ROOT) attaches a retrieved branch directly to the root alongside original candidates. This is easy to deploy because the root token is known early and enables parallel retrieval. However, it demands extra candidate slots. Under a fixed verification budget, these slots displace strong candidates near the root and dilute the retrieval benefit. Alternatively, **Graft**(TAIL) appends retrieval to the ends of branches. This avoids root-level competition but introduces a strict prefix dependency. The retrieved suffix is only helpful if the entire preceding draft prefix is accepted. A single early rejection renders the tail unreachable.

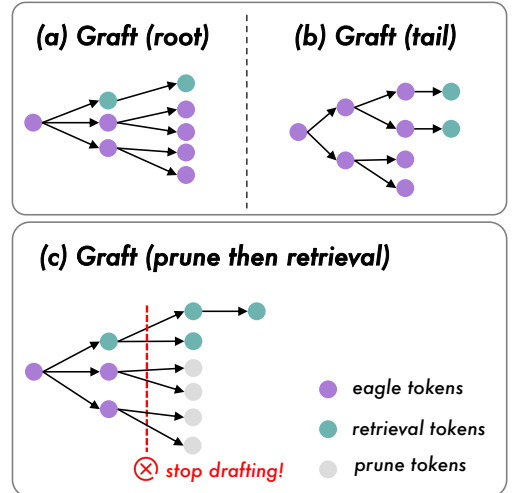


Figure 3 | Comparison of three retrieval insertion strategies.

Figure 2 isolates these limitations by constraining all static-tree variants to the same total candidate budget. **Graft** (ROOT) falls below EAGLE3 on average because it displaces too many high-quality draft nodes. **Graft** (TAIL) performs slightly better but yields only marginal and task-dependent gains. These observations dictate that retrieval should not simply sit beside or behind the dense tree. Instead, it must be inserted **exactly where pruning creates space**. **Pruning supplies the budget** that retrieval typically lacks. **Retrieval repairs the coverage lost by pruning** and mitigates the MAT penalty from over-pruning. We formalize this budget compensation as follows. After pruning, **Graft** grafts retrieved candidates into the freed topological space:

$$\mathcal{T}_G = \mathcal{T}_\pi \cup \mathcal{G}, \quad |\mathcal{T}_G| \leq |\mathcal{T}_E|, \quad \mathcal{T}_G \not\subseteq \mathcal{T}_E \text{ in general.} \quad (4)$$

By incorporating nodes outside the original subtree, **Graft** bypasses the dynamic-tree constraint. It preserves the low latency of pruned drafting while recovering candidate coverage via cheap retrieved continuations. This successfully breaks the pruning-only Pareto frontier. The core motivation of **Graft** is to prune unreliable drafting to release budget and then graft retrieval candidates into those exact slots. This repairs coverage loss within a single and fixed-budget target verification pass.

3. Graft: Hybrid Tree Construction for Speculative Decoding

Graft follows a simple principle: *draft less when autoregressive drafting becomes unreliable, and retrieve more with the budget released by pruning*. Although our implementation uses EAGLE-3 (Li et al., 2025) as the base tree drafter, the design only assumes a fixed-budget tree-style speculative verifier. Instead of expanding the full static draft tree at every decoding step, **Graft** first applies pruning to low-confidence draft branches. It then grafts retrieval-based branches into the released slots and verifies the merged tree with the unchanged target-model verification rule. The final tree keeps the original verification budget but reallocates capacity from uncertain drafting to cheap context-aware continuations. As summarized in Figure 4, the method has three coupled components. Section 3.1 releases budget through calibrated pruning. Section 3.2 spends that budget through retrieval grafting, and Section 3.3 verifies the hybrid tree while updating the retrieval state.

3.1. Dynamic-Depth Pruning for Budget Release

The first challenge, illustrated in Figure 4(a), is deciding how far to expand the base draft tree without introducing excessive decision error. Dense dynamic-tree methods like Dynamic Depth Decoding (DDD) (Brown et al., 2024) make fine-grained decisions at many intermediate depths. While this control can remove wasted candidates, confidence signals at most intermediate layers remain noisy. Frequent decisions can accumulate over-pruning and prematurely remove valid branches. **Graft** therefore uses a small set of calibrated pruning checkpoints $d \in \mathcal{D}_{\text{prune}}$. Similar to the calibration strategy in ECHO (Hu et al., 2026), we select these checkpoints at depths where confidence is highly discriminative. The objective fundamentally differs: **Graft** uses pruning strictly to release budget for retrieval rather than to reshape the draft tree again.

For a node j at draft depth d , we define its cumulative path score as:

$$S_{d,j} = S_{d-1,\mathcal{F}(j)} + \log q(x_{d,j} | h_{d-1,\mathcal{F}(j)}), \quad (5)$$

where q is the draft model’s output distribution and $\mathcal{F}(j)$ denotes the parent node of j . The highest-scoring path defines the confidence at depth d :

$$c_d = \exp \left(\max_j S_{d,j} \right). \quad (6)$$

At each checkpoint $d \in \mathcal{D}_{\text{prune}}$, **Graft** evaluates this confidence against a calibrated threshold τ_d :

$$g_d = \mathbb{1}[c_d > \tau_d]. \quad (7)$$

If $g_d = 0$, **Graft** prunes the draft tree at this checkpoint and skips deeper draft expansion.

The policy is intentionally biased toward shallow and reliable pruning decisions. We set stricter thresholds at shallow checkpoints so that uncertain branches are pruned before drafting errors compound. This design reduces the cumulative misjudgment risk of dense control and creates more retrieval budget exactly when the

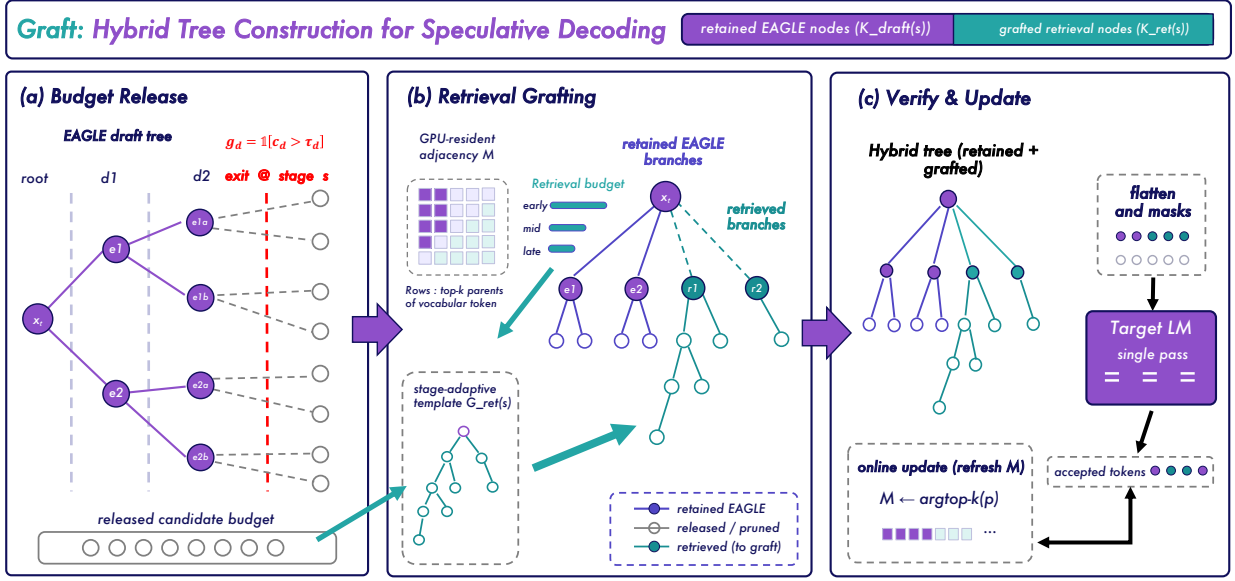


Figure 4 | **Overview of Graft.** (a) **Budget release:** dynamic-depth pruning removes unreliable draft-tree nodes and frees candidate slots under the same verification budget. (b) **Retrieval grafting:** root-centered retrieval runs in parallel with drafting, selects a stage-adaptive template, and fills the released slots with retrieved candidates. (c) **Verification and update:** retained draft nodes and grafted retrieval nodes are flattened into one hybrid tree for a single target-model verification pass, after which verified nodes refresh the GPU-resident retrieval matrix.

draft model proves least trustworthy. Let $K_{\max}$ denote the verification budget of the original static base tree. For a pruning stage s , Graft decomposes this fixed budget as:

$$K_{\max} = K_s^{\text{draft}} + K_s^{\text{ret}}, \quad (8)$$

where K_s^{draft} is the number of retained draft-tree nodes and K_s^{ret} is the budget assigned to retrieval. Shallower pruning stages allocate fewer draft nodes and grant a larger budget to retrieval, while deeper pruning stages preserve more of the original draft tree. Dynamic-depth pruning therefore reshapes the composition of the fixed candidate budget without inflating the target-side verification cost.

3.2. Retrieval-Grafted Tree Construction

After pruning releases candidate slots, the next question is how to spend them without creating another serial bottleneck, as shown in Figure 4(b). Prior retrieval-based speculative decoding methods often build candidates through prompt lookup, suffix matching, or external datastore search (Saxena, 2023; He et al., 2024; Liu et al., 2025b; Hu et al., 2025), whose cost is paid on the critical path. Graft instead uses a GPU-resident transition matrix and a root-centered retrieval template. Once the current root token is known, retrieval can start without waiting for deeper draft nodes; after the pruning stage is known, Graft selects the stage-matched prefix of this retrieved template and grafts it into the released slots. This makes retrieval a budget-compensation module inside fixed-budget tree construction, rather than a standalone drafter.

GPU-resident adjacency matrix. Graft maintains an adjacency matrix

$$\mathcal{M} \in \mathcal{V}^{|\mathcal{V}| \times k}, \quad (9)$$

where each row stores the top- k candidate successors for a vocabulary token. The storage cost is $O(|\mathcal{V}|k)$ token IDs, where k is small in practice. The matrix is stored directly on GPU, so generating a retrieved node reduces to a single row-column gather from its parent token. Compared with CPU-side tries, hash tables, or dynamically synchronized datastores, this representation avoids host-device communication and irregular control flow during decoding. This property is crucial for serving systems, where small synchronization overheads can erase the gains of speculation under high concurrency.

Lightweight parallel retrieval. This matrix design makes retrieval cheap enough to serve as budget compensation rather than a second parametric drafter. For a retrieval template $\mathcal{G}_s^{\text{ret}}$ with N_s retrieved nodes, depth d_s , and maximum layer width B_s , the total lookup work is $O(N_s) \leq O(B_s d_s)$. Since nodes at the same depth are independent once their parents are known, they are generated by batched GPU gathers, making the critical path scale with d_s instead of N_s . Although this cost is small compared with transformer-based drafting, placing it after pruning would still introduce a serial stage in every decoding round. Graft therefore materializes the root-centered retrieval envelope in parallel with tree drafting and only selects the prefix that matches the released budget. This preserves the latency saving from pruning while giving retrieval enough room to recover candidate coverage.

Stage-adaptive retrieval templates. The retrieval template is designed to fit the static base-tree envelope. Let D_E and W_E denote the maximum depth and maximum width of the original base tree. By default, the retrieval template uses the same maximum depth and width, so retrieved nodes follow a topology compatible with the tree verification path and later matrix updates. During decoding, Graft first prepares a full root-centered retrieval envelope and then selects a stage-specific sub-template after pruning determines how many slots have been released. Let $\mathcal{G}_s^{\text{ret}}$ denote the retrieval sub-template for pruning stage s . Starting from the root token x_r , each retrieved node u is generated by indexing the matrix row of its parent:

$$x_u = \mathcal{M}[x_{\mathcal{F}(u)}, r_u], \quad (10)$$

where $\mathcal{F}(u)$ denotes the parent of node u , and r_u is the successor rank specified by $\mathcal{G}_s^{\text{ret}}$. Concretely, if pruning happens at the root stage d_0 , most of the original tree budget is released, so Graft keeps only a small set of reliable draft nodes and fills a large retrieval branch. If pruning happens after an intermediate stage such as d_1 , fewer slots are released and Graft uses a medium retrieval branch. If the draft tree survives until a deeper checkpoint such as d_5 , only a small retrieval branch is grafted; if no pruning is triggered, the retrieval branch is empty or minimal. In our implementation with a 60-node verification budget, these stages keep 8, 24, and 40 draft nodes and assign 52, 36, and 20 slots to retrieval, respectively.

The templates are static within each stage and intentionally imbalanced. High-rank successors receive more children and extend deeper, forming a strong greedy continuation chain, while lower-rank successors receive fewer descendants to preserve sibling coverage without wasting budget. This follows the same intuition as Token Recycling (Luo et al., 2024): spend more verification budget on high-yield successor ranks while retaining enough width for alternatives. Appendix A.1 provides the exact template distributions used in our implementation.

Finally, Graft merges the retained draft nodes and retrieved nodes into a single hybrid tree:

$$\mathcal{T}_s = \mathcal{T}_s^{\text{draft}} \cup \mathcal{G}_s^{\text{ret}}, \quad |\mathcal{T}_s| = K_{\text{max}}. \quad (11)$$

This equation makes the role of grafting explicit: retrieval changes the candidate identities inside the tree, but the total verification budget remains fixed.

3.3. Hybrid Verification and Online Update

The final stage, shown in Figure 4(c), must satisfy two requirements: it must preserve lossless speculative verification, and it must keep the retrieval source aligned with the current context. Graft handles both requirements in the verification stage. The target model verifies retained draft nodes and grafted retrieval nodes uniformly, and the resulting target distributions are used to refresh the GPU-resident adjacency matrix.

Hybrid-tree verification. After grafting, all candidate tokens in $\mathcal{T}_s$ are flattened into a one-dimensional token list according to their parent-child relations. From this list, Graft rebuilds the standard tree position IDs, ancestor mask, and candidate-path indices, and then sends the whole hybrid tree to the target model in one parallel verification pass. Because $|\mathcal{T}_s| = K_{\text{max}}$, the target-side tree-attention interface and mask shape are unchanged from the base tree verifier; only the token identities occupying part of the tree are replaced by retrieved candidates. The acceptance rule is also unchanged. Retrieved tokens are proposals rather than committed outputs, and every output token must still be accepted by the target model under the standard speculative decoding rule. Therefore, Graft preserves the lossless property of speculative decoding without modifying the target model or the tree-attention kernel.

Warm-up and online update. A remaining issue for retrieval is cold start. Methods such as Prompt Lookup Decoding (PLD) (Saxena, 2023) rely heavily on tokens already present in the prompt, so their retrieval space is limited at the beginning of generation. Graft mitigates this limitation with a lightweight warm-up phase, using the same warm-up protocol as the base tree-drafter evaluation. Warm-up serves two roles: it initializes the adjacency matrix with prior successor patterns, and it calibrates the pruning checkpoints and thresholds $\{\tau_d\}$ following the same spirit as ECHO. As a result, Graft combines static prior knowledge with dynamic information from the current user prompt and recent generation history.

During generation, the matrix is continuously refreshed using the target model’s output distributions over verified tree nodes:

$$\mathcal{M}[\tilde{x}_i] = \text{argtop } k(\tilde{p}_{i+1}), \quad (12)$$

where $\tilde{x}_i$ is a verified draft token and $\tilde{p}_{i+1}$ is the target next-token distribution at that position. The update uses the whole verified tree, including both accepted and rejected nodes, so rejected candidates can still contribute useful target-guided successor information. Tree verification is particularly suitable for this update rule: each verification step evaluates many candidate nodes under different prefixes, providing rich transition signals for the adjacency matrix at almost no extra cost. As decoding proceeds, the retrieval matrix becomes increasingly aligned with the current context.

Long-context adaptation. Graft further adapts grafted retrieval to long-context serving. When the prefix becomes longer, each draft branch must attend to a larger history, so pruning saves a larger fraction of draft-side overhead. At the same time, long prompts and verified tree nodes expose more local transition patterns, which makes the GPU-resident retrieval matrix more informative before and during generation. We therefore initialize the retrieval state with warm-up and prompt-prefill signals, and then keep refreshing it through online tree verification. For this long-context instantiation, we adopt a YaRN-adapted 64K EAGLE-3 checkpoint following the long-context setting in SpecPV (Tan et al., 2025), since the public tree-drafter checkpoints we use are mainly trained around 2K context windows. For execution, Graft uses chunked prefilling for long prompts and scaled dot-product attention (SDPA) for tree verification, making the retrieval module suitable for long-context industrial serving without CPU-side retrieval or extra target-model forward passes.

Overall, Graft couples pruning, retrieval grafting, and target-guided update. Dynamic-depth pruning releases budget from unreliable autoregressive drafting; retrieval grafting repopulates that budget with cheap context-aware candidates; hybrid verification preserves losslessness and keeps the retrieval matrix aligned with the target model.

4. Experiments

4.1. Experimental Setting

Datasets and models. We evaluate Graft under three complementary settings: short-context generation, long-context generation, and high-concurrency serving. For short-context evaluation, we follow the protocols of EAGLE-3 (Li et al., 2025) and Spec-Bench (Xia et al., 2024) on five benchmarks covering code generation, mathematical reasoning, summarization, and open-ended chat: HumanEval (Chen et al., 2021), GSM8K (Cobbe et al., 2021), CNN/DM (Nallapati et al., 2016), Alpaca (Taori et al., 2023), and MT-Bench (Zheng et al., 2023). We evaluate a diverse set of target models, including Vicuna-13B (Chiang et al., 2023), LLaMA-3.1-8B (Grattafiori et al., 2024), and Qwen3-8B/32B/235B (Yang et al., 2025a). For long-context evaluation, we use QMSum, GovReport, MultiNews, LCC, and RepoBench-P from LongBench (Bai et al., 2024a,b), covering meeting summarization, report summarization, multi-document summarization, code completion, and repository-level code completion. Long-context experiments are conducted on LLaMA-3.1-8B and Qwen3-4B/8B/14B with EAGLE3-64K draft modules, following the YaRN-adapted EAGLE setting used in SpecPV (Tan et al., 2025). We also include a length-scaling study from 4K to 32K context tokens to evaluate whether the retrieval matrix benefits from richer prompt evidence.

Baselines and implementation. We benchmark Graft against representative methods across four categories of speculative decoding: (1) standard speculative decoding (Chen et al., 2023); (2) retrieval-based methods, including Lookahead (Fu et al., 2024), PLD (Saxena, 2023), SAMD (Hu et al., 2025), and TR (Luo et al., 2024); (3) training-based methods, including Medusa (Cai et al., 2024) and EAGLE-3 (Li et al., 2025); and (4) dynamic-tree methods, including DDD (Brown et al., 2024) and ECHO (Hu et al., 2026). For long-context experiments,



Table 1 | Main results on five benchmarks under the short-context setting. Performance comparison between Graft and existing baselines across diverse model configurations. Bold numbers denote the best speedup and MAT.

Models	Methods	HumanEval		GSM8K		CNN/DM		Alpaca		MT-Bench		Avg.
		MAT	Speedup	MAT	Speedup	MAT	Speedup	MAT	Speedup	MAT	Speedup	
Vicuna-13B	Lookahead	1.75	1.64×	1.89	1.75×	1.51	1.46×	1.47	1.49×	1.65	1.61×	1.59×
	Sps	2.53	1.78×	1.95	1.71×	2.26	1.69×	2.01	1.76×	2.26	1.79×	1.75×
	Medusa	2.78	2.21×	2.64	2.15×	2.05	1.63×	2.36	1.89×	2.61	2.13×	2.00×
	PLD	1.91	1.85×	1.97	1.75×	2.31	2.45×	1.26	1.20×	1.55	1.60×	1.77×
	SAMD	2.51	2.25×	2.47	1.91×	2.71	2.05×	2.23	1.87×	2.15	1.74×	1.96×
	TR	2.73	2.09×	2.68	1.99×	2.88	2.24×	2.49	1.95×	2.37	1.82×	2.02×
	DDD	8.21	4.97×	6.23	3.94×	6.07	3.48×	5.96	3.57×	6.07	3.99×	3.99×
	ECHO	8.35	5.15×	6.51	3.97×	6.22	3.49×	6.28	3.69×	6.35	4.05×	4.07×
	EAGLE3-2K	8.42	4.80×	6.81	3.83×	6.42	3.42×	6.52	3.67×	6.81	3.85×	3.91×
	Graft	8.53	5.41×	6.85	4.13×	6.51	3.63×	6.54	3.79×	6.93	4.13×	4.20×
LLaMA3.1-8B	TR	2.56	2.03×	2.48	1.94×	2.65	2.14×	1.94	1.87×	1.87	1.76×	1.95×
	ECHO	7.12	4.13×	6.28	4.07×	5.24	3.18×	6.83	4.13×	6.23	3.77×	3.86×
	EAGLE3-2K	7.19	3.98×	6.50	3.94×	5.47	3.05×	7.09	4.06×	6.42	3.61×	3.73×
	Graft	7.28	4.28×	6.53	4.17×	5.65	3.31×	7.12	4.20×	6.49	3.88×	3.97×
Qwen3-8B	TR	2.45	1.97×	2.56	1.98×	2.67	2.13×	2.31	1.79×	2.36	1.86×	1.95×
	ECHO	3.85	2.44×	3.82	2.33×	3.18	2.23×	3.32	2.19×	3.68	2.18×	2.27×
	EAGLE3	3.91	2.28×	3.92	2.17×	3.25	1.95×	3.44	2.09×	3.71	2.03×	2.10×
	Graft	3.97	2.58×	4.08	2.53×	3.41	2.37×	3.56	2.29×	3.79	2.36×	2.43×
	Δ ($\uparrow$, %)		$\uparrow 13.2\%$		$\uparrow 16.6\%$		$\uparrow 21.5\%$		$\uparrow 9.6\%$		$\uparrow 16.3\%$	$\uparrow 15.3\%$
Qwen3-32B	TR	2.43	1.91×	2.48	1.93×	2.53	1.97×	2.29	1.74×	2.37	1.83×	1.88×
	ECHO	2.82	2.36×	3.26	2.67×	2.51	1.98×	2.76	2.25×	2.96	2.27×	2.31×
	EAGLE3	2.96	2.14×	3.34	2.46×	2.55	1.82×	2.78	2.14×	3.02	2.03×	2.12×
	Graft	3.11	2.51×	3.54	2.86×	2.79	2.19×	2.93	2.41×	3.15	2.46×	2.49×
	Δ ($\uparrow$, %)		$\uparrow 17.3\%$		$\uparrow 16.3\%$		$\uparrow 20.3\%$		$\uparrow 12.6\%$		$\uparrow 21.2\%$	$\uparrow 17.4\%$
Qwen3-235B	TR	2.31	1.85×	2.17	1.75×	2.23	1.77×	1.96	1.66×	2.05	1.72×	1.75×
	ECHO	2.30	2.13×	2.58	1.88×	2.01	1.64×	2.21	1.92×	2.38	2.02×	1.92×
	EAGLE3-2K	2.39	1.81×	2.74	1.73×	2.02	1.45×	2.32	1.77×	2.56	1.81×	1.71×
	Graft	2.42	2.24×	2.85	2.06×	2.21	1.83×	2.44	2.13×	2.61	2.18×	2.09×
	Δ ($\uparrow$, %)		$\uparrow 23.8\%$		$\uparrow 19.1\%$		$\uparrow 26.2\%$		$\uparrow 20.3\%$		$\uparrow 20.4\%$	$\uparrow 21.8\%$

we additionally compare against EAGLE3-64K and long-context speculative decoding baselines, including MagicDec (Sadhukhan et al., 2024), TokenSwift (Wu et al., 2025), and TriForce (Sun et al., 2024). Some baselines only provide official implementations or released draft checkpoints for specific model families, mainly LLaMA3.1-8B. Therefore, we report each baseline on the compatible model settings supported by its public implementation, while keeping the target model, decoding configuration, and tree budget matched whenever possible. All experiments are conducted on 8 NVIDIA H20 GPUs with greedy decoding unless otherwise specified. The main short-context and long-context results are implemented in HuggingFace transformers; the high-concurrency ablations are implemented in SGLang to measure serving throughput under batched execution. For consistency across baselines, we implement all attention operations in long-context experiments using PyTorch scaled dot product attention. All baselines are reproduced using their official configurations, with detailed hyperparameters provided in Appendix C.

Metrics. Since Graft follows the target-model verification rule and does not modify the target model weights, we focus on acceleration metrics rather than generation quality. **Speedup** measures the actual speedup ratio relative to vanilla autoregressive decoding. For long-context experiments, speedup is computed using decoding time only, so all methods are compared after the shared long-prompt prefill stage. **mean accepted length (MAT)** measures the average number of tokens accepted per verification step. In the length-scaling ablation, we also report decoding throughput in tokens per second.

4.2. Main Results

Short-context case. Table 1 summarizes the performance of Graft in the short-context setting. Graft achieves consistent acceleration across all evaluated model families, with speedups ranging from 1.83× to 5.41×. The results demonstrate three distinct advantages: (1) **Acceleration over Static Trees.** Compared with EAGLE-3, Graft improves the average speedup across all model groups: 7.4% on Vicuna-13B, 5.9% on



Table 2 | **Main results for the long-context setting on five benchmarks.** We evaluate **Graft** against EAGLE3-64K on five long-context benchmarks using four model configurations. Bold numbers denote the best speedup and MAT.

Models	Methods	QMSum		GovReport		MultiNews		LCC		RepoBench-P		Avg.
		MAT	Speedup	MAT	Speedup	MAT	Speedup	MAT	Speedup	MAT	Speedup	
LLaMA3.1-8B	MagicDec	2.02	0.67×	2.05	0.68×	2.31	0.71×	2.57	0.82×	2.61	0.80×	0.74×
	TokenSwift	1.72	1.57×	1.74	1.58×	1.83	1.62×	1.85	1.82×	1.83	1.78×	1.67×
	Triforce	2.31	1.67×	2.41	1.64×	2.53	1.85×	2.67	2.05×	2.51	1.92×	1.81×
	EAGLE3-64K	6.02	2.97×	5.81	2.84×	5.83	2.85×	6.18	2.92×	6.13	3.02×	2.92×
	Graft	6.09	3.14×	5.95	3.09×	5.97	3.16×	6.39	3.37×	6.34	3.32×	3.22×
	Δ ($\uparrow$, %)		$\uparrow 5.7\%$		$\uparrow 8.8\%$		$\uparrow 10.9\%$		$\uparrow 15.4\%$		$\uparrow 9.9\%$	$\uparrow 10.3\%$
Qwen3-4B	EAGLE3-64K	3.32	1.67×	3.12	1.44×	2.83	1.55×	3.78	2.02×	3.67	1.72×	1.68×
	Graft	3.39	1.84×	3.23	1.68×	3.07	1.94×	4.00	2.39×	4.14	2.12×	1.99×
	Δ ($\uparrow$, %)		$\uparrow 10.2\%$		$\uparrow 16.7\%$		$\uparrow 25.2\%$		$\uparrow 18.3\%$		$\uparrow 23.3\%$	$\uparrow 18.7\%$
Qwen3-8B	EAGLE3-64K	3.50	1.66×	3.79	1.59×	3.39	1.68×	3.09	1.48×	3.01	1.27×	1.54×
	Graft	3.55	1.82×	3.87	1.77×	3.50	1.92×	3.57	1.96×	3.71	1.76×	1.85×
	Δ ($\uparrow$, %)		$\uparrow 9.6\%$		$\uparrow 11.3\%$		$\uparrow 14.3\%$		$\uparrow 32.4\%$		$\uparrow 38.6\%$	$\uparrow 20.2\%$
Qwen3-14B	EAGLE3-64K	3.18	1.71×	3.36	1.83×	3.01	1.71×	3.71	2.12×	3.61	1.95×	1.86×
	Graft	3.27	1.91×	3.46	2.09×	3.22	2.08×	3.93	2.46×	3.94	2.33×	2.17×
	Δ ($\uparrow$, %)		$\uparrow 11.7\%$		$\uparrow 14.2\%$		$\uparrow 21.6\%$		$\uparrow 16.0\%$		$\uparrow 19.5\%$	$\uparrow 16.6\%$

LLaMA3.1-8B, **15.7%** on Qwen3-8B, **17.5%** on Qwen3-32B, and **21.8%** on Qwen3-235B. Under the same total tree budget, **Graft** also keeps MAT comparable or higher, improving average MAT by **3.2%** on Qwen3-8B, **6.4%** on Qwen3-32B, and **3.3%** on Qwen3-235B. These gains come from both sides of the hybrid tree: pruning reduces draft cost, while retrieval fills the released budget with useful continuations rather than simply shrinking the candidate tree. The improvement in both speed and MAT indicates that the two components are mutually reinforcing: pruning gives retrieval enough room to affect the tree, and retrieval prevents the pruned tree from paying a large accepted-length penalty.

(2) Retrieval over Pruning Alone. Compared with ECHO, which mainly relies on dynamic-depth pruning, **Graft** is consistently faster and accepts more tokens. The average speedup improves over ECHO by **3.2%** on Vicuna-13B, **2.9%** on LLaMA3.1-8B, **7.0%** on Qwen3-8B, **7.8%** on Qwen3-32B, and **8.9%** on Qwen3-235B. At the same time, MAT increases by **4.5%**, **4.0%**, **5.4%**, **7.7%**, and **8.3%**, respectively. This confirms that retrieval effectively compensates for over-pruning by restoring plausible branches exactly where uncertain autoregressive drafting is pruned.

(3) Scalability to Large Models. **Graft** remains effective on the large-scale Qwen3-235B model, where the average accepted length is small and speculation is harder. It reaches a **2.09×** average speedup, with an average relative gain of **21.8%** over EAGLE3-2K (**1.71×**), **19.4%** over TR (**1.75×**), and **8.9%** over ECHO (**1.92×**). The improvement over EAGLE3-2K is consistent across all five benchmarks and reaches up to **26.2%** on CNN/DM. This validates the robustness of hybrid tree construction when pure autoregressive drafting has limited acceptance headroom.

Long-context case. Table 2 reports the decoding performance of **Graft** on long-context summarization and code completion benchmarks. **Graft** achieves speedups from **1.76×** to **3.37×**, and consistently improves over EAGLE3-64K under the same verification budget. The results reveal three key observations: **(1) Lower Draft Cost under Long Context.** Long-context decoding makes speculative drafting less forgiving. Prior long-context SD studies show that KV-cache access and attention become central bottlenecks in this regime (Tan et al., 2025; Yang et al., 2025b). As the prefix grows, both draft-side cache traffic and target-side tree verification become more expensive, so a low-confidence draft branch wastes more than it does in short-context generation. EAGLE3-64K extends the draft module to long windows, but it still spends draft-model computation on a fixed static tree before the target model verifies it. **Graft** changes this allocation: pruning removes unreliable draft expansion first, and the released budget is reused by retrieval without increasing the final verification size. Compared with EAGLE3-64K, **Graft** improves the average speedup by **10.3%** on LLaMA3.1-8B, **18.7%** on Qwen3-4B, **20.2%** on Qwen3-8B, and **16.6%** on Qwen3-14B. Meanwhile, the average MAT also increases by **2.6%**, **6.6%**, **8.5%**, and **5.6%**, respectively. The simultaneous gain in speed and MAT shows that **Graft** is not merely pruning the tree. It reduces expensive autoregressive drafting while preserving useful candidate coverage through retrieval.

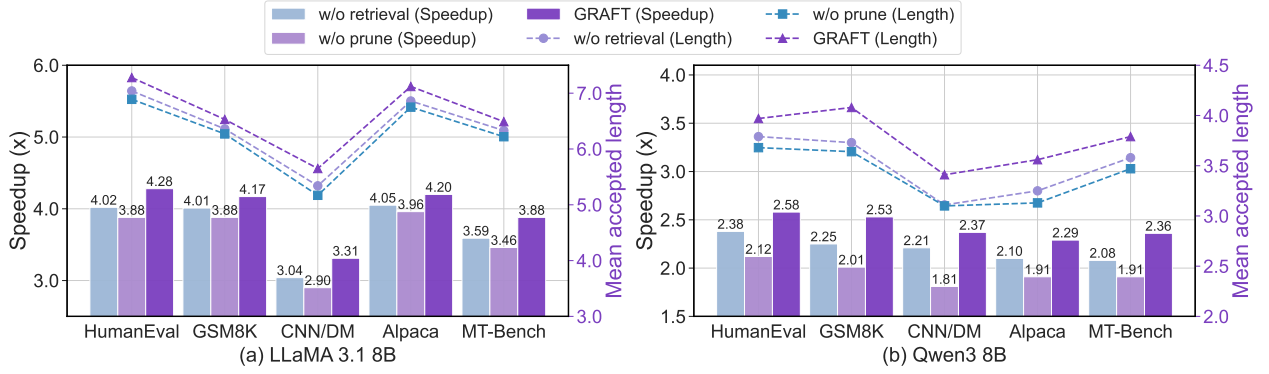


Figure 5 | **Component analysis of Graft.** Bars report speedup and dashed lines report MAT on LLaMA3.1-8B and Qwen3-8B. All variants keep the same total candidate budget. “w/o retrieval” keeps pruning but removes the grafted retrieval branch, testing whether pruning alone is sufficient. “w/o prune” uses a fixed draft/retrieval budget split, testing retrieval without confidence-adaptive budget release. The full **Graft** couples the two: pruning releases budget for retrieval, and retrieval compensates pruning-induced candidate loss.

(2) Context-Rich Retrieval Signals. Long contexts provide a much richer source of repeated phrases, document-specific entities, and local code patterns. This directly benefits the adjacency matrix because the retrieval library is not a static prompt-only cache. During prefilling and subsequent verification, **Graft** updates the matrix with target-model predictions from the verified draft tree, including both accepted and rejected draft nodes. Thus, a long prompt provides more initial evidence, and every tree verification step supplies many context-specific transitions for later root-centered retrieval. The effect is especially clear on code completion tasks: on Qwen3-8B, **Graft** improves speedup over EAGLE3-64K by **32.4%** on LCC and **38.6%** on RepoBench-P. On Qwen3-14B, the corresponding gains remain **16.0%** and **19.5%**. This is the regime where retrieval is most useful: repository-level code and long documents contain repeated local structures, so a GPU-resident transition matrix can recover high-quality continuations at much lower cost than another draft-model pass.

(3) Kernel-Friendly Long-Context Deployment. On LLaMA3.1-8B, **Graft** also outperforms long-context acceleration baselines by a clear margin. It reaches a **3.22×** average speedup, compared with **0.74×** for MagicDec, **1.67×** for TokenSwift, **1.81×** for TriForce, and **2.92×** for EAGLE3-64K. The improvement over TokenSwift and TriForce reaches **92.8%** and **77.9%**, respectively. The gap comes from a deployment-level difference. Long-context methods that rely on sparse or partial KV drafting can reduce attention cost, but they often introduce extra cache selection, synchronization, or target-like draft computation. **Graft** keeps retrieval as a GPU matrix lookup, overlaps it with autoregressive drafting, and then packs the retrieved branch back into the same tree-attention verification path. This makes the method compatible with optimized long-context attention implementations while avoiding an additional CPU-side retrieval path or an extra target forward pass.

4.3. Ablation Studies

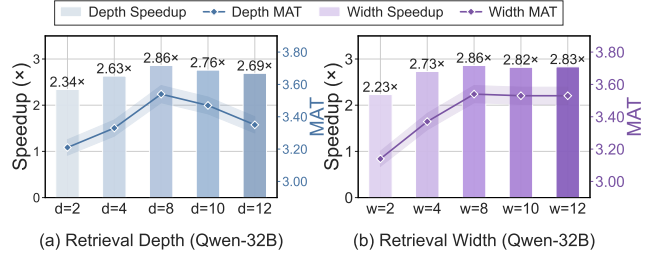
Component Analysis We first isolate the two core components of **Graft**: confidence pruning and retrieval grafting. To ensure a fair comparison, all variants use the same total candidate budget, so the difference comes from how the budget is allocated rather than how many tokens are verified. The *w/o retrieval* variant keeps pruning but removes the retrieval graft, leaving the released budget uncompensated. The *w/o prune* variant keeps both draft-tree candidates and retrieved candidates, but fixes their budget split across all decoding steps instead of dynamically reallocating budget according to confidence. As shown in Figure 5, removing either component degrades both speedup and MAT. On LLaMA3.1-8B, **Graft** improves the average speedup from **3.74×** to **3.97×** over *w/o retrieval*, and from **3.62×** to **3.97×** over *w/o prune*. On Qwen3-8B, the gain is more pronounced, increasing the average speedup from **2.20×** to **2.43×** over *w/o retrieval*, and from **1.95×** to **2.43×** over *w/o prune*. This shows that pruning and retrieval play complementary roles rather than acting as two independent optimizations. Without retrieval, pruning saves latency but leaves over-pruning uncompensated, so MAT remains lower. Without pruning, retrieval is restricted to a fixed small share of the tree and cannot receive enough budget when the draft model is uncertain. A fixed split also cannot track request difficulty: it may spend retrieval budget when the base drafter is already reliable, yet leave insufficient retrieval capacity when draft expansion should be pruned. The full method resolves both issues by letting pruning expose the right amount of retrieval budget and letting retrieval repair the coverage loss caused by pruning. The full

**Table 3 | Ablation on context length scaling under the long-context setting.** We evaluate **Graft** against EAGLE3-64K and existing baselines using four model configurations. Bold numbers denote the best speedup and MAT.

Models	Methods	4K			8K			16K			32K			Avg.
		MAT	Speed	Speedup	MAT	Speed	Speedup	MAT	Speed	Speedup	MAT	Speed	Speedup	
LLaMA3.1-8B	MagicDec	1.82	27.65	0.67×	1.88	22.53	0.71×	1.87	16.05	0.75×	1.88	9.83	0.75×	0.74×
	TokenSwift	1.62	56.54	1.37×	1.63	45.06	1.42×	1.65	32.94	1.58×	1.73	21.23	1.75×	1.53×
	Triforce	2.32	65.21	1.58×	2.48	54.88	1.73×	2.53	43.37	2.08×	2.51	27.90	2.30×	1.92×
	EAGLE3-64K	5.94	111.13	2.68×	5.95	87.44	2.75×	6.05	59.24	2.88×	6.01	35.78	2.95×	2.82×
	Graft	6.03	116.59	2.83×	6.04	93.05	2.93×	6.21	64.39	3.09×	6.20	38.58	3.19×	3.01×
	Δ ($\uparrow$, %)			$\uparrow 5.6\%$			$\uparrow 6.5\%$			$\uparrow 7.3\%$			$\uparrow 8.1\%$	$\uparrow 6.7\%$
Qwen3-4B	EAGLE3-64K	3.50	72.28	1.63×	3.52	59.65	1.69×	3.55	36.21	1.76×	3.61	24.94	1.83×	1.73×
	Graft	3.57	80.87	1.83×	3.59	66.45	1.92×	3.68	41.17	1.99×	3.73	28.43	2.08×	1.96×
	Δ ($\uparrow$, %)			$\uparrow 12.3\%$			$\uparrow 13.6\%$			$\uparrow 13.1\%$			$\uparrow 13.7\%$	$\uparrow 13.3\%$
Qwen3-8B	EAGLE3-64K	3.49	62.15	1.50×	3.54	51.96	1.69×	3.53	33.32	1.85×	3.56	23.52	1.94×	1.75×
	Graft	3.57	68.35	1.65×	3.63	57.78	1.91×	3.67	38.19	2.12×	3.68	27.34	2.26×	1.99×
	Δ ($\uparrow$, %)			$\uparrow 10.0\%$			$\uparrow 13.0\%$			$\uparrow 14.6\%$			$\uparrow 16.5\%$	$\uparrow 13.7\%$
Qwen3-14B	EAGLE3-64K	3.13	45.98	1.84×	3.19	39.39	1.93×	3.25	24.95	1.99×	3.24	18.18	2.02×	1.95×
	Graft	3.22	52.15	2.08×	3.28	45.08	2.21×	3.37	28.96	2.31×	3.43	22.17	2.38×	2.25×
	Δ ($\uparrow$, %)			$\uparrow 13.0\%$			$\uparrow 14.5\%$			$\uparrow 16.1\%$			$\uparrow 17.8\%$	$\uparrow 15.4\%$

Table 4 | Impact of warm-up rounds K on storage and speedup (Qwen3-32B).

Rounds (K)	Storage (MB)	Wall-time Speedup		
		HumanEval	GSM8K	MT-Bench
0	0.0	2.37×	2.71×	2.31×
1	0.13	2.41×	2.75×	2.36×
3	0.24	2.47×	2.79×	1.88×
5	0.37	2.51×	2.86×	2.46×
10	0.63	2.54×	2.91×	2.54×
25	1.45	2.63×	2.98×	2.66×
50	2.32	2.68×	3.03×	2.75×

**Figure 6 | Retrieval template depth and width ablation.** Bars report speedup and lines report MAT on Qwen3-32B when varying the maximum retrieval depth d or maximum retrieval width w . The default setting follows the base-tree envelope and balances long-chain continuation with sibling coverage.

design therefore achieves higher speed and longer accepted sequences under the same verification budget.

Context length scaling. Table 3 studies the effect of context length by grouping and truncating QMSum samples into 4K, 8K, 16K, and 32K buckets. Since all buckets come from the same task, the comparison largely isolates the impact of context length from dataset-specific differences. The trend is consistent with our design: as the context becomes longer, absolute decoding speed decreases for all methods due to heavier attention and KV-cache access, but the relative advantage of **Graft** becomes larger. Averaged across the four models, the speedup gain over EAGLE3-64K increases from 10.2% at 4K to 14.0% at 32K. The trend is more pronounced on larger Qwen models: on Qwen3-8B, the gain grows from 10.0% to 16.5%; on Qwen3-14B, it grows from 13.0% to 17.8%. This is intuitive for **Graft**. Longer contexts make fixed autoregressive drafting more costly, while also giving the online adjacency matrix more prompt evidence and more repeated local transitions to retrieve from. The MAT improvement follows the same direction, e.g., Qwen3-14B increases from 3.22 at 4K to 3.43 at 32K, showing that retrieval improves accepted length rather than only reducing overhead.

Warm-up Start Warm-up serves two purposes in **Graft**. First, it initializes the GPU-resident adjacency matrix with prior successor patterns before online generation begins. The warm-up data comes from ShareGPT, a separate conversation corpus commonly used for draft-model training, and is disjoint from all evaluation datasets, so it does not introduce data contamination. This alleviates the cold-start limitation of prompt-only retrieval methods such as PLD (Saxena, 2023), whose retrieval source is sparse at the beginning of decoding. As shown in Table 4, five warm-up rounds improve speedup from 2.37×



Table 5 | **Performance under stochastic decoding ($T=1$) on five benchmarks.** Performance comparison between **Graft** and existing baselines across diverse model configurations. Bold numbers denote the best speedup and MAT.

Models	Methods	HumanEval		GSM8K		CNN/DM		Alpaca		MT-Bench		Avg.
		MAT	Speedup	MAT	Speedup	MAT	Speedup	MAT	Speedup	MAT	Speedup	
LLaMA3.1-8B	ECHO	6.33	3.68×	5.34	3.53×	4.32	2.71×	6.17	3.51×	5.34	3.18×	3.32×
	EAGLE3	6.31	3.55×	5.45	3.36×	4.67	2.57×	6.23	3.34×	5.51	3.01×	3.17×
	Graft	6.41	3.81×	5.51	3.61×	4.83	2.83×	6.31	3.64×	5.57	3.25×	3.43×
	Δ ($\uparrow$, %)		$\uparrow 7.3\%$		$\uparrow 7.4\%$		$\uparrow 10.1\%$		$\uparrow 9.0\%$		$\uparrow 8.0\%$	$\uparrow 8.2\%$
Qwen3-8B	ECHO	3.21	2.21×	3.21	2.28×	2.67	1.99×	2.93	2.01×	3.28	2.04×	2.11×
	EAGLE3	3.23	1.87×	3.35	1.97×	2.83	1.71×	2.98	1.83×	3.34	1.89×	1.85×
	Graft	3.32	2.37×	3.43	2.43×	2.91	2.17×	3.06	2.17×	3.41	2.18×	2.26×
	Δ ($\uparrow$, %)		$\uparrow 26.7\%$		$\uparrow 23.4\%$		$\uparrow 26.9\%$		$\uparrow 18.6\%$		$\uparrow 15.3\%$	$\uparrow 22.2\%$

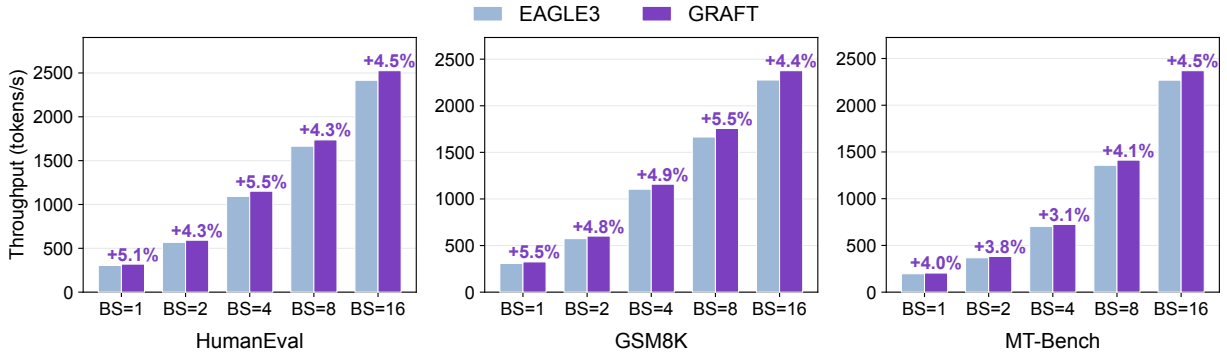


Figure 7 | **High-batch serving sanity check in SGLang on Qwen3-8B.** We compare EAGLE3 and **Graft** across batch sizes on three benchmarks. Bold numbers denote the best MAT and throughput.

thresholds are stable across datasets and that fixed thresholds transferred from HumanEval only incur minor degradation, indicating that the warm-up calibration generalizes well across tasks.

Effect of Retrieval Depth and Width We further analyze how the retrieval template should allocate its fixed budget between depth and width. By default, the maximum retrieval depth and width are aligned with the original base tree. As detailed in Appendix A.1, the default template is a Token-Recycling-inspired static and unbalanced tree: high-rank successors are assigned more children and a deeper greedy chain, while lower-rank successors keep limited width. This makes the retrieved branch compatible with the tree shape used during warm-up and online updates: the adjacency matrix is initialized and refreshed from transitions produced by the same tree topology, and even steps without pruning can still provide useful update signals. During decoding, **Graft** does not always use the full retrieval template. Instead, it selects a stage-specific template according to the pruning depth, so shallow pruning receives more retrieval slots while deeper pruning preserves more draft nodes.

Figure 6 shows that both overly narrow and overly aggressive templates are suboptimal. When the retrieval depth is too small, e.g., $d = 2$, the retrieved branch can only provide short continuations, yielding **2.34×** speedup and **3.21** MAT. Increasing the depth to $d = 8$ improves both metrics to **2.86×** and **3.54**, but pushing the chain deeper causes the benefit to decline because distant retrieved tokens become less reliable and consume budget that could have covered alternative branches. The width ablation follows the same pattern. Small width under-explores plausible successors, while excessive width spends too much budget on low-rank candidates; the best balance again appears around $w = 8$, where **Graft** reaches **2.86×** speedup and **3.54** MAT. These results support our template design: retrieval needs enough width and depth to compensate pruning, but under a fixed verification budget the shape should remain close to the base-tree envelope rather than becoming too chain-like or too flat.

4.4. Discussions

Temperature sampling. Table 5 evaluates **Graft** under stochastic decoding with temperature $T = 1$. Higher temperature makes the target distribution less concentrated, which usually lowers draft confidence and increases



Table 6 | **Performance over DFLASH on five benchmarks.** Performance comparison between **Graft** and existing baselines across diverse model configurations. Bold numbers denote the best speedup and MAT.

Models	Methods	HumanEval		GSM8K		CNN/DM		Alpaca		MT-Bench		Avg.
		MAT	Speedup	MAT	Speedup	MAT	Speedup	MAT	Speedup	MAT	Speedup	
Qwen3-8B	EAGLE3(60)	3.23	1.87×	3.35	1.97×	2.83	1.71×	2.98	1.83×	3.34	1.89×	1.85×
	Graft-EAGLE3(60)	3.32	2.37×	3.43	2.43×	2.91	2.17×	3.06	2.17×	3.41	2.18×	2.26×
	DFLASH(16)	6.43	5.07×	6.45	4.97×	3.83	2.23×	3.11	2.05×	4.39	2.69×	3.40×
	Graft-DFLASH(16)	6.61	5.37×	6.64	5.25×	4.23	2.68×	3.34	2.33×	4.54	2.93×	3.71×
	Δ over DFLASH ($\uparrow$, %)		$\uparrow$ 5.9%		$\uparrow$ 5.6%		$\uparrow$ 20.2%		$\uparrow$ 13.7%		$\uparrow$ 8.9%	$\uparrow$ 9.1%

the chance of early rejection. Even in this harder setting, **Graft** remains clearly better than the baselines. On LLaMA3.1-8B, **Graft** reaches a **3.43×** average speedup, outperforming EAGLE3 by **8.2%** and ECHO by **3.3%**. On Qwen3-8B, the advantage is larger: **Graft** achieves **2.26×** average speedup, improving over EAGLE3 by **22.2%** and ECHO by **7.1%**. The MAT is also consistently the highest across all five benchmarks. These results indicate that pruning and retrieval grafting are not tied to greedy decoding, and remain effective when token probabilities become flatter.

High-Batch Scalability Figure 7 provides a Qwen3-8B sanity check for **Graft** in SGLang serving across batch sizes from 1 to 16. In this setting, the benefit of dynamic depth is hard to realize directly, because batched serving favors fixed CUDA graphs and batch-aligned verification shapes. Changing the token budget of each request would require intrusive scheduler and kernel adaptations. Therefore, our high-batch implementation keeps the same gating criterion but applies it as pruning: low-confidence draft nodes are removed from the static tree, while the released slots are filled by a retrieval tree rooted at the current token. In other words, the high-concurrency version of **Graft** keeps the per-request verification budget unchanged, and grafts retrieval candidates onto the pruned base tree instead of dynamically changing tree depth. Despite this lightweight implementation, **Graft** consistently improves both throughput and MAT over EAGLE3. Across all evaluated batch sizes and benchmarks, **Graft** reports higher throughput and higher MAT than EAGLE3, showing that retrieval grafting remains useful even when the serving system keeps a fixed verification budget. We note that this SGLang implementation does not yet fully optimize retrieval kernels or scheduling, so the high-concurrency potential of grafted retrieval remains an important future direction. Detailed throughput numbers and implementation details are provided in Appendix D.

More Discussions. Due to space constraints, we provide additional derivations, system details, and supplementary analyses in the Appendix. Appendix B gives the theoretical analysis of pruning regret, retrieval coverage, speedup conditions, and lossless verification. Appendix A.1 details the retrieval template design, while Appendix C.4 reports cross-task threshold generalization. Appendix D provides the Qwen3-8B high-batch SGLang implementation and throughput table, and Appendix E profiles the runtime overhead of retrieval, merge, rebuild, verification, and online matrix updates. We further discuss the DFlash demo implementation in Appendix F and provide a broader related-work discussion in Appendix G.

4.5. Future Works

Grafting for block drafters. An immediate direction is to extend **Graft** to block drafters such as DFlash (Chen et al., 2026). DFlash drafts a block of tokens in one parallel diffusion pass, which substantially reduces drafting latency compared with autoregressive tree drafting. However, even with a block size of 16, its average accepted length is still typically around 5 to 7 tokens, and the gain over EAGLE-3 can be limited on harder chat benchmarks such as MT-Bench. We suspect that this limitation comes from a structural mismatch: DFlash performs bidirectional denoising within a block, whereas the target model verifies tokens under the original autoregressive factorization. Consequently, some positions in the drafted block may have low verification utility even though the entire block is generated efficiently.

This observation makes DFlash a natural testbed for grafting. In particular, the DFlash implementation produces token-level draft logits before target verification, which allows us to estimate the confidence of each drafted token and identify low-utility positions for pruning. Our observations further suggest that confidence derived from DFlash draft logits is positively correlated with token acceptance, making confidence-guided block-level pruning a natural next step. Instead of changing the block size or increasing the serving budget, one

can prune low-confidence diffusion tokens and graft retrieval-based candidates into the released positions. This preserves the parallel drafting advantage of DFlash while using retrieval to compensate for the block modeling mismatch, especially in scenarios where local repetition or prompt evidence is strong.

To examine this direction, we conduct a preliminary test by applying Graft on top of DFlash. As shown in Table 6, grafting consistently improves DFlash across all five benchmarks on Qwen3-8B. Specifically, Graft-DFLASH increases the average speedup from $3.40\times$ to $3.71\times$, corresponding to a relative improvement of 9.1%. The gains are particularly clear on CNN/DM and Alpaca, where the speedup improves from $2.23\times$ to $2.68\times$ and from $2.05\times$ to $2.33\times$, respectively. These preliminary results suggest that the computation saved by pruning low-confidence block tokens can be effectively repurposed for retrieval-based grafting, allowing Graft to partially recover the acceptance rate lost by block-level pruning without changing the serving budget. We provide the demo implementation and Graft (TAIL) strategy in Appendix F.

Hybrid drafting for RL rollouts. Another promising scenario is large scale RL rollout, especially group sampling settings such as GRPO (Shao et al., 2024). In this setting, maintaining a separate static draft model is costly and fragile, because the target policy keeps changing during training and the draft distribution can quickly drift away. Pure retrieval methods, such as Seer (Qin et al., 2025), avoid this overhead by mining local n-gram overlap from responses in the same rollout group, but their hit rate can be unstable when high temperature sampling or task difficulty reduces cross response similarity. Recent rollout acceleration systems increasingly treat speculative decoding, including MTP heads, as a stable primitive for RL generation (Iso et al., 2026; Liu et al., 2025a; Qin et al., 2025; Wang et al., 2025). Multi token prediction (MTP) (Gloeckle et al., 2024) is attractive here because it is tied to the current target model and does not depend on cross response overlap. However, its long range prediction quality can still be affected by rollout distribution shift, and its acceleration ceiling is limited when the accepted prefix remains short. This suggests a natural extension of Graft: combine an MTP branch with an online n-gram retrieval branch in a grafted draft tree. The MTP branch uses the target model’s internal features to provide a stable multi token prediction path, so it gives a reliable throughput floor even during cold start or low overlap rollout groups. The n-gram branch, in contrast, dynamically aggregates group generation history and can retrieve long high quality continuations when rollout trajectories share reasoning templates, tool use patterns, or local solution structures. In such a hybrid design, MTP stabilizes the lower bound while retrieval raises the upper bound. Confidence pruning can then decide which MTP positions are unreliable and graft retrieved candidates into those slots. This is particularly attractive for Agentic RL, where tail requests often dominate iteration time and where a robust acceleration base must avoid extra model memory while still exploiting the strong local repetition created by grouped rollouts.

5. Related Works

Speculative Decoding Speculative decoding (SD) accelerates LLM inference through the draft-then-verify paradigm, where candidate tokens are proposed by a fast drafter and verified in parallel by the target model (Leviathan et al., 2023; Chen et al., 2023). Existing methods improve this pipeline from several directions: separate lightweight draft models (Leviathan et al., 2023; Chen et al., 2023), auxiliary-head or self-speculative drafters such as Medusa and EAGLE (Cai et al., 2024; Li et al., 2025), multi-token prediction (Gloeckle et al., 2024), and tree-based verification that explores multiple continuations in one target forward (Miao et al., 2024a). These works mainly ask how to generate more draft candidates. Graft instead focuses on how to spend a fixed candidate budget: low-confidence draft branches are pruned, and the released slots are filled with retrieval candidates.

Retrieval-based Speculative Decoding Retrieval-based SD is appealing because it can propose candidate tokens with little additional draft-model computation while preserving the lossless verification guarantee. Representative methods retrieve candidates from prompt-local matches, external datastores, cached transitions, or distributional matching logic, including Lookahead Decoding, PLD, REST, Token Recycling, LogitSpec, and SAMD (Fu et al., 2024; Saxena, 2023; He et al., 2024; Luo et al., 2024; Liu et al., 2025b; Hu et al., 2025; Shen et al., 2026). However, most retrieval methods are used as standalone drafters and remain tied to CPU-side lookup, prompt-local evidence, or task-specific data structures. As a result, their speedup ceiling is often limited, and their adaptation to large models, high-concurrency serving, and tree-style verification is still underexplored. Graft uses retrieval in a different role. Rather than replacing the parametric tree drafter, it grafts retrieval branches into the budget released by pruning, stores token transitions in a GPU-friendly adjacency matrix, and



packs the retrieved candidates back into the same tree-attention verification path. This makes retrieval better aligned with industrial serving requirements.

Dynamic Token Tree Tree-based SD expands multiple candidate paths to increase the accepted length (Miao et al., 2024a). Static tree methods such as EAGLE-3 use fixed structures with low overhead, but inevitably spend budget on uncertain deep branches (Li et al., 2025). Dynamic token tree methods adapt the topology according to token probability or confidence (Li et al., 2024b; Zhang et al., 2024b; Brown et al., 2024; Hu et al., 2026; Zhang et al., 2024a). They improve budget utilization, but dense control can introduce misjudgment accumulation and irregular request shapes that are hard to deploy efficiently in serving kernels. Graft keeps the efficient static-tree verification interface while introducing lightweight pruning and retrieval grafting. The final verification budget remains consistent with the original tree, avoiding complex ragged-batch adaptation.

Speculative Decoding for Long Context. As context length increases, large language model inference becomes heavily memory-bound due to escalating KV-cache traffic and attention overhead, a bottleneck that inherently degrades speculative decoding. Recent methods attempt to mitigate this issue through sparse or partial KV caching, hierarchical speculation, and long-context draft adaptation (Sun et al., 2024; Yang et al., 2025b; Wu et al., 2025). However, these solutions primarily optimize the KV-cache dimension and lack tight integration with fixed-budget tree verification. Furthermore, frameworks such as SpecPV (Tan et al., 2025) and the retrieval-accelerated RAPID (Chen et al., 2025) rely on lossy approximations, thereby sacrificing the exact equivalence guarantee of standard speculative decoding. In contrast, Graft instantiates grafting on a long-context tree drafter and uses retrieval to exploit richer local transition evidence while strictly maintaining efficient, lossless tree verification.

6. Conclusion

This paper investigates whether the budget released by dynamic tree pruning can be reused to break the latency-MAT trade-off in tree-based speculative decoding. We approach the question from two directions: a preliminary analysis showing that pruning-only dynamic trees remain bounded by the dense base tree, and a hybrid construction that uses retrieval to refill the slots made available by pruning.

Our analysis suggests that pruning should not be viewed only as candidate removal. It is also a budget allocation decision. When low-confidence draft branches are pruned, spending the saved slots on the same uncertain drafter often yields limited returns. Retrieval offers a complementary source of candidates because it can reuse prompt, history, and verification signals with little draft-model computation. The key is to use retrieval as compensation rather than as a standalone drafter: root insertion competes with strong draft-tree candidates, and tail insertion depends on a fully accepted draft prefix, while grafting places retrieved candidates exactly where pruning creates space.

The experiments support this picture across short-context, long-context, and high-batch settings. Graft improves speed while maintaining or increasing MAT, showing that it is not merely shrinking the tree. In short-context benchmarks, it reaches up to $5.41\times$ speedup and improves over EAGLE-3 most strongly on large models such as Qwen3-235B. In long-context generation, the benefit becomes more pronounced because tree drafting becomes more expensive and the retrieval matrix receives richer local transition evidence. The SGLang results further show that grafting remains useful under fixed per-request budgets, where fully dynamic tree depth is difficult to deploy. Beyond the tree drafter used in our main implementation, our preliminary study on DFlash suggests that the same compensation principle may also extend to block drafting paradigms. By pruning low-confidence diffusion tokens and grafting retrieval-based candidates into the released positions, Graft improves the average DFlash speedup from $3.40\times$ to $3.71\times$ on Qwen3-8B, offering initial evidence that grafting can help compensate for the mismatch between bidirectional block denoising and autoregressive verification.

There are still limits to this first instantiation. Retrieval is most effective when the prompt or generation history contains useful local transition structure, and our current high-concurrency implementation does not yet fully optimize retrieval kernels or scheduler-level placement. The DFlash exploration is also preliminary: block drafters differ from tree drafters in topology and confidence calibration, and a more systematic design of confidence-guided block pruning and retrieval placement remains future work. These limitations point to a broader direction: speculative decoding should separate *which* draft candidates are worth pruning from *how* the released budget should be repopulated. Graft takes a step in this direction by showing that drafting less and retrieving more can form a practical, lossless, and serving-friendly path beyond the pruning frontier.



References

- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3119–3137, Bangkok, Thailand, August 2024a. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.172. URL <https://aclanthology.org/2024.acl-long.172>.
- Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. *arXiv preprint arXiv:2412.15204*, 2024b.
- Oscar Brown, Zhengjie Wang, Andrea Do, Nikhil Mathew, and Cheng Yu. Dynamic depth decoding: Faster speculative decoding for llms. *arXiv preprint arXiv:2409.00142*, 2024.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- Guanzheng Chen, Qilong Feng, Jinjie Ni, Xin Li, and Michael Qizhe Shieh. Rapid: Long-context inference with retrieval-augmented speculative decoding. *arXiv preprint arXiv:2502.20330*, 2025.
- Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block Diffusion for Flash Speculative Decoding. *arXiv preprint arXiv:2602.06036*, 2026.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023. URL <https://lmsys.org/blog/2023-03-30-vicuna/>.
- Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. Pact: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*, 2018.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.
- Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of llm inference using lookahead decoding. *arXiv preprint arXiv:2402.02057*, 2024.
- Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, and Anirudh Goyal et al. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.



- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Zhenyu He, Zexuan Zhong, Tianle Cai, Jason Lee, and Di He. Rest: Retrieval-based speculative decoding. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 1582–1595, 2024.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Xinyi Hu, Yuhao Shen, Baolin Zhang, Hengxin Zhang, Jun Dai, Shuang Ge, Lei Chen, Yue Li, and Mingcheng Wan. Echo: Elastic speculative decoding with sparse gating for high-concurrency scenarios. *arXiv preprint arXiv:2604.09603*, 2026.
- Yuxuan Hu, Ke Wang, Xiaokang Zhang, Fanjin Zhang, Cuiping Li, Hong Chen, and Jing Zhang. Sam decoding: Speculative decoding via suffix automaton. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12187–12204, 2025.
- Hayate Iso, Tiyasa Mitra, Sudipta Mondal, Rasoul Shafipour, Venmugil Elango, Terry Kong, Yuki Huang, Seonjin Na, Izzy Putterman, Benjamin Chislett, Maor Ashkenazi, Joseph Guman, Gerald Shen, Tugrul Konuk, Ashwath Aithal, Ritika Borkar, Ran Zilberstein, and Bitu Rouhani. Accelerating rl post-training rollouts via system-integrated speculative decoding. *arXiv preprint arXiv:2604.26779*, 2026.
- Yicheng Ji, Jun Zhang, Heming Xia, Jinpeng Chen, Lidan Shou, Gang Chen, and Huan Li. SpecVLM: Enhancing speculative decoding of video LLMs via verifier-guided token pruning. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 7205–7219, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.366. URL <https://aclanthology.org/2025.emnlp-main.366/>.
- Yicheng Ji, Jun Zhang, Jinpeng Chen, Cong Wang, Lidan Shou, Gang Chen, and Huan Li. See the forest for the trees: Loosely speculative decoding via visual-semantic guidance for efficient inference of video llms, 2026. URL <https://arxiv.org/abs/2604.05650>.
- Quan Kong, Yuhao Shen, Yicheng Ji, Huan Li, and Cong Wang. Parallellvm: Lossless video-llm acceleration with visual alignment aware parallel speculative decoding. *arXiv preprint arXiv:2603.19610*, 2026.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286. PMLR, 2023.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024a.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-2: Faster inference of language models with dynamic draft trees. *arXiv preprint arXiv:2406.16858*, 2024b.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025.
- Bingshuai Liu, Ante Wang, Zijun Min, Liang Yao, Haibo Zhang, Yang Liu, Xu Han, Peng Li, Anxiang Zeng, and Jinsong Su. Spec-rl: Accelerating on-policy reinforcement learning with speculative rollouts. *arXiv preprint arXiv:2509.23232*, 2025a.
- Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, and Winston Hu. Parallel speculative decoding with adaptive draft length. *arXiv preprint arXiv:2408.11850*, 2024.
- Tianyu Liu, Qitan Lv, Hao Li, Xing Gao, and Xiao Sun. Logitspec: Accelerating retrieval-based speculative decoding via next next token speculation. *arXiv preprint arXiv:2507.01449*, 2025b.
- Tianyu Liu, Qitan Lv, Yuhao Shen, Xiao Sun, and Xiaoyan Sun. Talon: Confidence-aware speculative decoding with adaptive token trees. *arXiv preprint arXiv:2601.07353*, 2026a.



- Tianyu Liu, Yuhao Shen, Xinyi Hu, Baolin Zhang, Hengxin Zhang, Jun Dai, Jun Zhang, Shuang Ge, Lei Chen, Yue Li, and MingCheng Wan. When hidden states drift: Can kv caches rescue long-range speculative decoding?, 2026b. URL <https://arxiv.org/abs/2604.26412>.
- Xianzhen Luo, Yixuan Wang, Qingfu Zhu, Zhiming Zhang, Xuanyu Zhang, Qing Yang, Dongliang Xu, and Wanxiang Che. Turning trash into treasure: Accelerating inference of large language models with token recycling. *arXiv preprint arXiv:2408.08696*, 2024.
- Jonathan Mamou, Oren Pereg, Daniel Korat, Moshe Berchansky, Nadav Timor, Moshe Wasserblat, and Roy Schwartz. Dynamic speculation lookahead accelerates speculative decoding of large language models. *arXiv preprint arXiv:2405.04304*, 2024.
- Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pp. 932–949, 2024a.
- Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pp. 932–949, 2024b.
- Ramesh Nallapati, Bowen Zhou, Caglar Gulcehre, Bing Xiang, et al. Abstractive text summarization using sequence-to-sequence rnns and beyond. *arXiv preprint arXiv:1602.06023*, 2016.
- Ruoyu Qin, Weiran He, Weixiao Huang, Yangkun Zhang, Yikai Zhao, Bo Pang, Xinran Xu, Yingdi Shan, Yongwei Wu, and Mingxing Zhang. Seer: Online context learning for fast synchronous llm reinforcement learning. *arXiv preprint arXiv:2511.14617*, 2025.
- Ranajoy Sadhukhan, Jian Chen, Zhuoming Chen, Vashisth Tiwari, Ruihang Lai, Jinyuan Shi, Ian En-Hsu Yen, Avner May, Tianqi Chen, and Beidi Chen. Magicdec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding. *arXiv preprint arXiv:2408.11049*, 2024.
- Apoorv Saxena. Prompt lookup decoding, November 2023. URL <https://github.com/apoorvumang/prompt-lookup-decoding/>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Yuhao Shen, Junyi Shen, Quan Kong, Tianyu Liu, Yao Lu, and Cong Wang. Speculative decoding via hybrid drafting and rollback-aware branch parallelism. *arXiv preprint arXiv:2506.01979*, 2025.
- Yuhao Shen, Tianyu Liu, Junyi Shen, Jinyang Wu, Quan Kong, Li Huan, and Cong Wang. Double: Breaking the acceleration limit via double retrieval speculative parallelism. *arXiv preprint arXiv:2601.05524*, 2026.
- Mingbo Song, Heming Xia, Jun Zhang, Chak Tou Leong, Qiancheng Xu, Wenjie Li, and Sujian Li. KNN-SSD: Enabling dynamic self-speculative decoding via nearest neighbor layer set optimization. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), *Findings of the Association for Computational Linguistics: EACL 2026*, pp. 641–655, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-386-9. doi: 10.18653/v1/2026.findings-eacl.31. URL <https://aclanthology.org/2026.findings-eacl.31/>.
- Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 31, 2018.
- Hanshi Sun, Zhuoming Chen, Xinyu Yang, Yuandong Tian, and Beidi Chen. Triforce: Lossless acceleration of long sequence generation with hierarchical speculative decoding. *arXiv preprint arXiv:2404.11912*, 2024.
- Zhendong Tan, Xingjun Zhang, Chaoyi Hu, Junjie Peng, and Kun Xia. Specpv: Improving self-speculative decoding for long-context generation via partial verification. *arXiv preprint arXiv:2512.02337*, 2025.



- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models*. <https://crfm.stanford.edu/2023/03/13/alpaca.html>, 3(6):7, 2023.
- Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Zhendong Li, Xiaoyang Li, et al. Reinforcement learning optimization for large-scale learning: An efficient and user-friendly scaling library. *arXiv preprint arXiv:2506.06122*, 2025.
- Tong Wu, Junzhe Shen, Zixia Jia, Yuxuan Wang, and Zilong Zheng. Tokenswift: Lossless acceleration of ultra long sequence generation. *arXiv preprint arXiv:2502.18890*, 2025.
- Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. *arXiv preprint arXiv:2401.07851*, 2024.
- Heming Xia, Yongqi Li, Jun Zhang, Cunxiao Du, and Wenjie Li. Swift: On-the-fly self-speculative decoding for llm inference acceleration, 2025. URL <https://arxiv.org/abs/2410.06916>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.
- Penghui Yang, Cunxiao Du, Fengzhuo Zhang, Haonan Wang, Tianyu Pang, Chao Du, and Bo An. Longspec: Long-context lossless speculative decoding with efficient drafting and verification. *arXiv preprint arXiv:2502.17421*, 2025b.
- Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft& verify: Lossless large language model acceleration via self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11263–11282. Association for Computational Linguistics, 2024a. doi: 10.18653/v1/2024.acl-long.607. URL <http://dx.doi.org/10.18653/v1/2024.acl-long.607>.
- Jun Zhang, Yicheng Ji, Feiyang Ren, Yihang Li, Bowen Zeng, Zonghao Chen, Ke Chen, Lidan Shou, Gang Chen, and Huan Li. Efficient inference for large vision-language models: Bottlenecks, techniques, and prospects, 2026. URL <https://arxiv.org/abs/2604.05546>.
- Situo Zhang, Hankun Wang, Da Ma, Zichen Zhu, Lu Chen, Kunyao Lan, and Kai Yu. Adaeagle: Optimizing speculative decoding via explicit modeling of adaptive draft structures. *arXiv preprint arXiv:2412.18910*, 2024b.
- Ziyin Zhang, Jiahao Xu, Tian Liang, Xingyu Chen, Zhiwei He, Rui Wang, and Zhaopeng Tu. Draft model knows when to stop: A self-verification length policy for speculative decoding. *arXiv preprint arXiv:2411.18462*, 2024c.
- Weilin Zhao, Yuxiang Huang, Xu Han, Wang Xu, Chaojun Xiao, Xinrong Zhang, Yewei Fang, Kaihuo Zhang, Zhiyuan Liu, and Maosong Sun. Ouroboros: Generating longer drafts phrase by phrase for faster speculative decoding. *arXiv preprint arXiv:2402.13720*, 2024a.
- Yao Zhao, Zhitian Xie, Chen Liang, Chenyi Zhuang, and Jinjie Gu. Lookahead: An inference acceleration framework for large language model with lossless generation accuracy. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 6344–6355, 2024b.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023.

**Algorithm 1** Graft Generation

-
- 1: **Input:** prefix $x_{1:t}$, target model p , tree drafter q , budget $K_{\max}$, adjacency matrix $\mathcal{M}$
 - 2: Ensure $\mathcal{M}$ is allocated on GPU; initialize empty rows if needed
 - 3: Precompute retrieval-template caches for pruning stages $s \in \{d_0, d_1, d_5\}$
 - 4: Run target prefill on $x_{1:t}$ and initialize the first draft tree
 - 5: Update $\mathcal{M}$ with prefill logits using Eq. 14
 - 6: **while** the sequence has not reached an end condition **do**
 - 7: Verify the current hybrid tree with the target model and obtain node logits
 - 8: Update $\mathcal{M}$ using target logits from all verified tree nodes
 - 9: Select the accepted path with the standard speculative decoding acceptance rule
 - 10: Append accepted tokens and update target KV cache, draft hidden state, and current root
 - 11: Build the next hybrid tree with Algorithm 2
 - 12: **end while**
 - 13: **Output:** generated sequence
-

Appendix Contents

- Appendix A: generation procedure, pseudocode, and retrieval-template details for Graft.
- Appendix B: theoretical analysis of pruning regret, retrieval coverage, speedup condition, and lossless verification.
- Appendix C: evaluation details, baseline settings, and cross-task threshold generalization.
- Appendix D: high-concurrency implementation details and SGLang throughput table.
- Appendix E: runtime profiling and overhead breakdown.
- Appendix F: DFlash demo implementation and Graft (TAIL) configuration.
- Appendix G: detailed discussion of related work.

A. Procedure and Pseudocode

This section summarizes the generation pipeline used by Graft. The implementation has three coupled parts: (i) confidence-based pruning over a fixed-budget draft tree, (ii) retrieval grafting from a GPU-resident adjacency matrix, and (iii) online updates of this matrix using target-model verification logits. The final output tree is always packed into the standard tree-attention verification path, so retrieval changes only the proposal set and not the target-model acceptance rule.

Stage-adaptive budget. Let $K_{\max}$ be the candidate budget of the original static base tree. Graft assigns this budget according to the pruning stage s :

$$K_{\max} = K_s^{\text{draft}} + K_s^{\text{ret}}. \quad (13)$$

In our implementation, pruning at checkpoints d_0 , d_1 , and d_5 keeps 8, 24, and 40 draft-tree nodes, respectively, and allocates the remaining slots to retrieval. If no pruning is triggered, Graft degenerates to the original base tree with no retrieval grafting. For each stage, we precompute a retrieval template with the same maximum width and depth as the corresponding base-tree shape. This makes initialization and online updates compatible with the tree topology: even decoding rounds without pruning still refresh useful transition rows in the adjacency matrix.

GPU-resident transition matrix. Graft maintains an adjacency matrix $\mathcal{M} \in \mathcal{V}^{|\mathcal{V}| \times k}$, where each row stores the top- k likely successors of a token. During prefill and every verification round, target logits from all verified tree nodes are used to update this matrix:

$$\mathcal{M}[\tilde{x}_i] \leftarrow \text{argtop } k(\tilde{p}_{i+1}), \quad (14)$$

where $\tilde{x}_i$ is a verified draft token and $\tilde{p}_{i+1}$ is the target next-token distribution at that node. This update uses both accepted and rejected candidates, since both have already been evaluated by the target model.

**Algorithm 2** Draft-and-Graft Tree Construction

```

1: Input: current root token  $x_t$ , draft hidden state, adjacency matrix  $\mathcal{M}$ , thresholds  $\{\tau_d\}$ , budget  $K_{\max}$ 
2: Run the root draft step and compute the root confidence
3: if root confidence is below  $\tau_0$  then
4:   Set pruning stage  $s = d_0$  and prune the draft tree at this depth
5: else
6:   for each draft expansion depth  $d$  do
7:     Expand one draft layer and rank nodes by cumulative path score
8:     if  $d$  is a pruning checkpoint and confidence is below  $\tau_d$  then
9:       Set pruning stage  $s = d$  and prune the draft tree at this depth
10:    end if
11:  end for
12: end if
13: Resolve  $(K_s^{\text{draft}}, K_s^{\text{ret}})$  according to Eq. 13
14: Keep the top  $K_s^{\text{draft}}$  draft nodes by cumulative path score
15: if no pruning is triggered or  $K_s^{\text{ret}} = 0$  then
16:   Use the retained draft nodes as the final proposal tree
17: else
18:   Traverse the cached retrieval template from root  $x_t$  by matrix lookup in  $\mathcal{M}$ 
19:   Append  $K_s^{\text{ret}}$  retrieved nodes to the retained draft nodes
20: end if
21: Build the final tree mask, tree position IDs, and candidate path indices
22: Return: hybrid proposal tree with at most  $K_{\max}$  candidate nodes

```

Table 7 | **Retrieval templates used by Graft**. The stage-specific templates follow the same static unbalanced branching principle as the full template: top-ranked successors receive more children, lower-ranked successors receive fewer, and the greedy chain can reach depth 9.

Template	Trigger	Retained Draft Nodes	Retrieved Nodes	Retrieval Depth Distribution
Full envelope	Ablation / design	–	80	8 + 16 + 14 + 11 + 8 + 7 + 6 + 5 + 5
d_0 template	Prune at root	8	52	8 + 10 + 8 + 6 + 5 + 4 + 4 + 4 + 3
d_1 template	Prune after depth 1	24	36	6 + 7 + 5 + 4 + 4 + 3 + 3 + 2 + 2
d_5 template	Prune after depth 5	40	20	4 + 3 + 3 + 2 + 2 + 2 + 2 + 1 + 1

A.1. Retrieval Template Details

The retrieval tree in **Graft** is implemented as a static, unbalanced template over successor ranks. Following the template construction idea in Token Recycling (Luo et al., 2024), higher-ranked successor paths receive more children and are extended deeper, while lower-ranked paths receive fewer descendants. This design matches the empirical observation that top-ranked retrieved successors are more likely to be accepted, but still keeps enough width to cover alternative local continuations.

Each template node is represented by a rank path $[r_1, \dots, r_d]$. At inference time, **Graft** fills the template in a BFS-like manner from the current root token x_t . For a node u with parent $\mathcal{F}(u)$ and successor rank r_u , the token is obtained by a GPU matrix lookup:

$$x_u = \mathcal{M}[x_{\mathcal{F}(u)}, r_u]. \quad (15)$$

After all retained draft nodes and retrieved nodes are collected, **Graft** rebuilds the tree mask, tree position IDs, and candidate-path indices, and sends the merged sequence to the original tree-attention verification path.

Table 7 lists the templates used in our implementation. The full template contains 80 non-root nodes across 9 retrieval depths and defines the default retrieval envelope used for template design and width/depth ablations. During decoding, **Graft** selects a stage-specific prefix according to the pruning location. Shallower pruning stages allocate more of the fixed budget to retrieval, while deeper stages preserve more draft nodes.

B. Theoretical Analysis

This appendix provides a more detailed analysis of the pruning tradeoff introduced in Sec. 2. We focus on three properties of Graft: (i) why pruning creates an acceptance regret relative to the dense tree, (ii) why retrieval grafting can recover part of this regret under the same verification budget, and (iii) why Graft remains lossless once the final hybrid tree is verified by the target model.

B.1. Pruning Regret

Let $\mathcal{T}_0$ denote the original dense draft tree and $\mathcal{T}_\pi \subseteq \mathcal{T}_0$ denote the sparse draft tree retained by a pruning policy π . For any draft tree $\mathcal{T}$, let $L(\mathcal{T})$ be the accepted prefix length after target verification and define $A(\mathcal{T}) = \mathbb{E}[L(\mathcal{T})]$. Since pruning only removes candidates, the accepted length of the pruned tree is upper bounded by that of the dense tree.

Proposition 1 (Monotonicity of Tree Verification). *If $\mathcal{T}_1 \subseteq \mathcal{T}_2$ and both trees are verified by the same target model under the same acceptance rule, then*

$$L(\mathcal{T}_1) \leq L(\mathcal{T}_2), \quad A(\mathcal{T}_1) \leq A(\mathcal{T}_2). \quad (16)$$

Proof. All candidate paths available in $\mathcal{T}_1$ are also available in $\mathcal{T}_2$. Target verification selects an accepted prefix among the candidate paths according to the same rule. Adding extra candidates cannot invalidate any previously valid prefix, so the accepted prefix length cannot decrease. Taking expectation gives the second inequality. $\square$

The acceptance regret of pruning is therefore

$$\mathcal{R}_\pi = A(\mathcal{T}_0) - A(\mathcal{T}_\pi) = \mathbb{E}[L(\mathcal{T}_0) - L(\mathcal{T}_\pi)] \geq 0. \quad (17)$$

This regret is the price paid for reducing draft latency. As shown by the ratio in Eq. 3, a pruned tree improves the speed proxy only when its latency saving is large enough to compensate for the MAT loss caused by pruning.

Over-pruning contribution. The regret in Eq. 17 is mainly caused by over-pruning. Let F_d be the event that the policy prunes at depth d even though the dense tree would accept beyond d . Then a lower bound on the regret is

$$\mathcal{R}_\pi \geq \sum_d \Pr(F_d) \mathbb{E}[L(\mathcal{T}_0) - d \mid F_d]. \quad (18)$$

Thus, dense control can be harmful even when each individual decision is only mildly noisy. If the over-pruning probability at depth d is ϵ_d , the probability of at least one harmful pruning decision along a path is

$$\Pr(\text{over-pruning}) = 1 - \prod_{d \in \mathcal{D}} (1 - \epsilon_d), \quad (19)$$

which increases as more decision points are introduced. This formalizes the misjudgment accumulation discussed in Sec. 2.

B.2. Coverage Gain from Retrieval Grafting

Pruning releases candidate slots. Let $\Delta K = |\mathcal{T}_0| - |\mathcal{T}_\pi|$ be the released budget, and let $\mathcal{G}$ be the retrieval tree grafted into these slots, with $|\mathcal{G}| \leq \Delta K$. The final hybrid tree is

$$\mathcal{T}_g = \mathcal{T}_\pi \cup \mathcal{G}, \quad |\mathcal{T}_g| \leq |\mathcal{T}_0|. \quad (20)$$

Therefore, Graft keeps the same verification budget as the dense tree while changing how the budget is populated.

Proposition 2 (Non-decreasing Acceptance after Grafting). *For a fixed pruned tree $\mathcal{T}_\pi$, adding grafted retrieval candidates cannot reduce the accepted length:*

$$A(\mathcal{T}_g) \geq A(\mathcal{T}_\pi). \quad (21)$$



Proof. By construction, $\mathcal{T}_\pi \subseteq \mathcal{T}_g$. The result follows directly from Proposition 1. $\square$

The useful part of grafting comes from improved coverage at the pruning frontier. Consider a node at depth d where draft expansion is pruned. Let $\mathcal{S}$ be the set of remaining draft-tree candidates at this frontier, and let $\mathcal{R}$ be the retrieved candidate set grafted at the same frontier. Under the target next-token distribution $p_t(\cdot \mid x_{<d})$, define the coverage of a candidate set $\mathcal{A}$ as

$$\text{Cov}(\mathcal{A}) = \sum_{x \in \mathcal{A}} p_t(x \mid x_{<d}). \quad (22)$$

Proposition 3 (Coverage Gain by Retrieval). *The grafted frontier has coverage gain*

$$\text{Cov}(\mathcal{S} \cup \mathcal{R}) - \text{Cov}(\mathcal{S}) = \sum_{x \in \mathcal{R} \setminus \mathcal{S}} p_t(x \mid x_{<d}) \geq 0. \quad (23)$$

The inequality is strict whenever the retrieved set contains at least one new token with non-zero target probability.

Proof. The result follows from the additivity of probability mass over the union of two finite candidate sets. Only retrieved tokens not already covered by $\mathcal{S}$ contribute additional mass. $\square$

Implication. Pure pruning reduces latency but leaves a coverage gap. Retrieval grafting uses the released slots to increase frontier coverage without invoking another draft-model pass. When the retrieved tokens match repeated local patterns in the prompt or recent generation history, the gain in Eq. 23 can recover part of the regret in Eq. 17.

B.3. Speedup Condition with Grafted Retrieval

Let $A_0 = A(\mathcal{T}_0)$, $A_\pi = A(\mathcal{T}_\pi)$, and $A_g = A(\mathcal{T}_g)$. Let

$$\Gamma_g = A_g - A_\pi \quad (24)$$

be the accepted-length gain brought by grafted retrieval. Since $|\mathcal{T}_g| \leq |\mathcal{T}_0|$, the verification cost of the hybrid tree is no larger than that of the dense tree under the same packing strategy. The hybrid tree improves over pure pruning when

$$\frac{(A_\pi + \Gamma_g + 1)}{T_{\text{draft}}(\mathcal{T}_\pi) + T_{\text{ret}} + T_{\text{verify}}(\mathcal{T}_g)} > \frac{(A_\pi + 1)}{T_{\text{draft}}(\mathcal{T}_\pi) + T_{\text{verify}}(\mathcal{T}_\pi)}. \quad (25)$$

In **Graft**, retrieval is implemented as GPU-resident matrix indexing and is overlapped with tree drafting, so T_{ret} is small. This makes the condition mainly depend on whether Γ_g is positive. Compared with the dense tree, **Graft** aims to preserve the latency benefit of pruning while reducing the regret term:

$$A_0 - A_g = \mathcal{R}_\pi - \Gamma_g. \quad (26)$$

Thus, grafting is useful whenever the retrieved candidates recover part of the removed valid continuation, i.e., $\Gamma_g > 0$.

B.4. Lossless Verification

Finally, we show that **Graft** does not change the output distribution of speculative decoding. The key point is that retrieval only changes the proposal set. All proposed tokens, whether generated by the parametric drafter or retrieved from the adjacency matrix, are verified by the target model using the same speculative acceptance rule.

Proposition 4 (Lossless Equivalence). *Assume the final hybrid tree $\mathcal{T}_g$ is verified by the target model with the standard speculative sampling rule. Then **Graft** produces the same output distribution as autoregressive decoding from the target model.*

Proof. Speculative sampling is lossless because every proposed token is accepted or rejected according to the target distribution, and a rejected position is resampled from the target correction distribution (Leviathan et al., 2023; Chen et al., 2023). This guarantee does not depend on how the proposal tokens are generated. In Graft, retrieval does not write tokens directly to the output sequence; it only supplies additional proposals inside $\mathcal{T}_g$. Since all proposals are passed through the same target verification rule, the resulting token distribution is identical to standard target-model autoregressive decoding. The same argument applies to the high-concurrency Qwen3-8B sanity check in SGLang: the implementation keeps the per-request verification budget and tree-attention shape fixed, and only replaces low-confidence proposal tokens with retrieved proposal tokens before the unchanged target verification step. Therefore, batching, pruning, and grafting affect proposal efficiency but not the target distribution. $\square$

Takeaway. The theoretical role of Graft is therefore simple: pruning reduces expensive tree drafting but creates acceptance regret; retrieval grafting uses the released candidate slots to improve coverage; target verification preserves losslessness.

C. Evaluation Details

For reproducibility, we provide additional details for the experimental setup in Sec. 4. The source code of this project will be made available at a later time.

C.1. Data Configurations

We evaluate Graft under three complementary settings: short-context generation, long-context generation, and batched high-concurrency decoding. For short-context evaluation, we follow the standard EAGLE-3 and Spec-Bench setup, covering code generation, mathematical reasoning, summarization, instruction following, and chat: HumanEval (Chen et al., 2021), GSM8K (Cobbe et al., 2021), CNN/DM (Nallapati et al., 2016), Alpaca (Taori et al., 2023), and MT-Bench (Zheng et al., 2023). The maximum generation length for these tasks is set to 1024 tokens.

For long-context evaluation, we use five generation-heavy benchmarks from LongBench (Bai et al., 2024a,b). **QMSum** evaluates meeting summarization, where the model must compress long meeting transcripts into concise summaries. **GovReport** evaluates long-document report summarization over government reports. **MultiNews** evaluates multi-document summarization, requiring the model to integrate information across multiple news articles. **LCC** evaluates long-context code completion, where the prefix contains extended source-code context. **RepoBench-P** evaluates repository-level code completion with broader project context. For these long-context tasks, all methods share the same prompt prefill, and we report decoding-only speedup to isolate the acceleration effect during token generation. In the length-scaling study, we group QMSum examples into different context-length buckets from 4K to 32K tokens. To make the comparison as controlled as possible, we sample from the same prompt pool whenever possible and truncate prompts to the target length while preserving contiguous context. When natural boundaries are available, we truncate at meeting-turn or sentence boundaries to maintain semantic continuity.

C.2. Model Configurations

For short-context experiments, we use the official EAGLE/ECHO-compatible checkpoints and configurations provided by the corresponding baselines. The target and draft model pairs used by the main short-context experiments are summarized in Table 8. All model weights are loaded in `bf16` format without quantization. As a training-free method, Graft does not modify any target or draft model parameters during evaluation.

For long-context experiments, we pair each target model with a YaRN-adapted EAGLE3-64K draft module, as summarized in Table 9.

Table 10 lists the key model architecture configurations used in our experiments. Vicuna 68M corresponds to the lightweight draft module used by the Vicuna-13B tree-draft pair.



Table 8 | **Short-context target and draft model checkpoints.** We follow the public EAGLE3-compatible checkpoints used by the corresponding baselines.

Model Group	Draft Model Checkpoint	Target Model Checkpoint
Vicuna-13B	yuhuili/EAGLE3-Vicuna1.3-13B	lmsys/vicuna-13b-v1.3
LLaMA3.1-8B	yuhuili/EAGLE3-LLaMA3.1-Instruct-8B	meta-llama/Llama-3.1-8B-Instruct
Qwen3-8B	AngelSlim/Qwen3-8B-eagle3	Qwen/Qwen3-8B
Qwen3-32B	AngelSlim/Qwen3-32B-eagle3	Qwen/Qwen3-32B
Qwen3-235B	lmsys/Qwen3-235B-A22B-EAGLE3	Qwen/Qwen3-235B-A22B

Table 9 | **Long-context target and draft model checkpoints.** Each target model is paired with the corresponding YaRN-adapted EAGLE3-64K draft module.

Target Model	Draft Model
LLaMA3.1-8B-Instruct	EAGLE3-LLaMA3.1-8B-YARN-64K
Qwen3-4B	EAGLE3-Qwen3-4B-YARN-64K
Qwen3-8B	EAGLE3-Qwen3-8B-YARN-64K
Qwen3-14B	EAGLE3-Qwen3-14B-YARN-64K

C.3. Detailed Baselines

Short-context baselines. We compare *Graft* with representative baselines from four families of speculative decoding. **Standard SD** includes the original draft-then-verify speculative sampling framework (Leviathan et al., 2023; Chen et al., 2023), reported as Sps in our tables. **Retrieval-based SD** includes Lookahead (Fu et al., 2024), PLD (Saxena, 2023), SAMD (Hu et al., 2025), and Token Recycling (TR) (Luo et al., 2024). These methods reuse prompt or generation history to propose candidates without training a new draft model. **Training-based methods** include Medusa (Cai et al., 2024) and EAGLE-3 (Li et al., 2025). EAGLE-3 is our primary static-tree instantiation because it provides a strong public tree drafter; *Graft* changes how the candidate budget is allocated rather than depending on an EAGLE-specific verification rule. **Dynamic-tree methods** include DDD (Brown et al., 2024) and ECHO (Hu et al., 2026), which reduce wasted drafting through adaptive tree construction. When a baseline was originally implemented for a weaker or older tree backbone, we adapt the scheduling policy to the strongest available public setting whenever possible, so that the comparison focuses on tree construction rather than draft-model strength. For short-context tree baselines, we follow the official configuration with depth 8, top- $k = 10$, and total candidate budget 60. All variants of *Graft* keep the same total verification budget as the matched EAGLE baseline.

Long-context baselines. For long-context experiments, we first compare with **EAGLE3-64K**, which serves as the direct long-context tree baseline under the same target model and context window. We also compare against representative long-context speculative decoding systems: **MagicDec**, **TokenSwift**, and **TriForce**. These methods mainly optimize long-context drafting through sparse KV cache, partial KV cache, or hierarchical speculation. Some of them only provide public implementations or released draft checkpoints for specific model families, mainly LLaMA3.1-8B. Therefore, we report each method on the compatible model settings supported by its official implementation, while keeping the target model, decoding configuration, and total candidate budget matched whenever possible. For high-concurrency experiments, we use the official SGLang tree-serving configuration and keep the per-request candidate budget fixed; *Graft* only prunes low-confidence draft nodes and grafts retrieval nodes into the released slots.

Implementation details. All experiments are conducted on $8 \times$ NVIDIA H20 GPUs with greedy decoding unless otherwise specified. The main short-context and long-context experiments are implemented with HuggingFace `transformers`, while high-concurrency ablations are implemented with SGLang to measure batched throughput. For consistency across long-context baselines, all attention operations are implemented using PyTorch scaled dot product attention. All baselines are reproduced using their official configurations whenever available.

Configuration protocols. For low-concurrency experiments ($BS = 1$), EAGLE-3, ECHO, and *Graft* use the same tree configuration with depth 8, top- $k = 10$, and total candidate budget 60. This setting maximizes the acceptance-length upper bound under the standard base-tree envelope. For high-concurrency experiments



Table 10 | Model architecture configurations.

Model	Layers	Dim.	FFN Dim.	Vocabulary Size
Vicuna 68M	2	768	3072	32000
Vicuna 13B	40	5120	13824	32000
LLaMA-3.1 8B	32	4096	14336	128256
Qwen3 8B	36	4096	12288	151936
Qwen3 32B	64	5120	25600	151936
Qwen3 235B-A22B	94	4096	12288	151936

Table 11 | **Calibration thresholds across datasets.** The calibrated pruning locations and gating thresholds are stable across different benchmarks, indicating that the sweet spots are mainly determined by the model capability rather than dataset-specific properties.

Gate	Calibrated threshold on Qwen3-8B					Range
	HumanEval	GSM8K	CNN/DM	Alpaca	MT-Bench	
d_0 (root)	0.15	0.14	0.12	0.14	0.14	0.03
d_1 (shallow)	0.13	0.14	0.15	0.15	0.13	0.02
d_5 (deep)	0.51	0.51	0.46	0.49	0.49	0.05

(BS > 1), we use the official SGLang tree-serving implementation and its default serving configuration. In this setting, Graft keeps the per-request verification budget unchanged and applies pruning plus retrieval grafting within the same static serving envelope. This avoids changing the CUDA-graph-compatible request shape while still replacing low-confidence draft nodes with retrieved candidates.

Timing protocol. Speedup is always computed against vanilla autoregressive decoding of the same target model under the same implementation stack. For short-context experiments, we report end-to-end decoding speedup under greedy decoding. For long-context experiments, prompt prefill is shared by all methods and excluded from the reported speedup; we compare only the decoding stage after prefill. Tokens/s is computed as the number of generated completion tokens divided by decoding wall time. This convention avoids conflating speculative decoding gains with differences in long-prompt prefilling cost.

Warm-up protocol. Warm-up data is drawn from ShareGPT, a separate conversation corpus commonly used in draft-model training. It is disjoint from all evaluation datasets used in our experiments, so it does not introduce data contamination. By default, we use five warm-up rounds. Warm-up initializes the GPU-resident adjacency matrix with prior successor patterns and calibrates the pruning checkpoints and thresholds. During online generation, matrix updates remain enabled and use target logits from verified tree nodes, including both accepted and rejected candidates.

C.4. Cross-Task Threshold Generalization

Warm-up also calibrates the pruning locations and gating thresholds used by Graft. The pruning locations are primarily determined by the intrinsic capability of the draft model, and are only weakly sensitive to the downstream dataset. As shown in Table 11, the calibrated gating thresholds remain stable across five benchmarks, with variations mostly within 0.05. Although different datasets may induce slightly different acceptance distributions, such small threshold fluctuations do not lead to significant performance degradation. Moreover, the retrieval component further compensates for small calibration mismatches by providing additional useful draft candidates.

To further evaluate robustness, we fix the thresholds calibrated on HumanEval and directly apply them to other datasets. As shown in Table 12, using fixed thresholds only causes a small relative degradation of about 1% – 3% compared with per-dataset calibration, while still substantially outperforming EAGLE3 on all benchmarks. These results indicate that Graft has strong robustness and generalization ability. In future work, an online threshold adaptation strategy based on sliding-window latency feedback could further improve deployment-time robustness.



Table 12 | **Robustness under fixed thresholds.** We fix the thresholds calibrated on HumanEval and apply them to other benchmarks. Fixed thresholds only incur minor degradation compared with per-dataset calibration, while still substantially outperforming EAGLE3.

Model	Method	GSM8K	CNN/DM	Alpaca	MT-Bench
Qwen3-8B	EAGLE3	3.92 / 2.17×	3.25 / 1.95×	3.44 / 2.09×	3.71 / 2.03×
	Graft	4.08 / 2.53×	3.41 / 2.37×	3.56 / 2.29×	3.79 / 2.36×
	Graft (fixed thresholds)	4.01 / 2.44× (−3%)	3.31 / 2.29× (−3%)	3.51 / 2.26× (−1%)	3.74 / 2.31× (−2%)

Table 13 | **High-batch sanity check in SGLang on Qwen3-8B.** We compare EAGLE3 and Graft across batch sizes on three benchmarks. Bold numbers denote the best MAT and throughput.

Benchmark	Methods	BS=1		BS=2		BS=4		BS=8		BS=16	
		MAT	Throughput	MAT	Throughput	MAT	Throughput	MAT	Throughput	MAT	Throughput
HumanEval	EAGLE3	3.10	303.97	3.10	567.82	3.09	1091.64	3.10	1664.65	3.10	2416.28
	Graft	3.22	319.54	3.21	592.18	3.22	1151.27	3.21	1736.25	3.24	2525.83
GSM8K	EAGLE3	3.15	307.42	3.15	573.86	3.14	1104.37	3.15	1664.62	3.15	2276.08
	Graft	3.27	324.31	3.26	601.47	3.27	1158.28	3.26	1756.23	3.28	2376.19
MT-Bench	EAGLE3	2.23	196.41	2.23	367.54	2.22	703.28	2.22	1357.01	2.23	2268.27
	Graft	2.35	204.27	2.34	381.63	2.35	724.85	2.34	1412.57	2.36	2369.43

D. High-Concurrency Implementation Details

This section complements the Qwen3-8B high-batch sanity results in Figure 7 and the discussion in Sec. 4.4. We implement the high-concurrency version of Graft on SGLang v0.5.4 using the `eagle_worker_v1` path. Different from the single-request implementation, the batched implementation does not change the per-request tree depth during drafting. Instead, all requests first finish the standard static tree construction for the current decoding round. After the whole batch is available, we apply depth-gated pruning to each request, replace the low-confidence draft slots with retrieved tokens, and graft the retrieved branch back into the same fixed-size candidate tensor. Thus, every request keeps the same total candidate budget as EAGLE3.

This design is intentionally simple. Compared with ECHO-style elastic budget scheduling, which reallocates different budgets to different requests, Graft does not require modifying CUDA-graph capture, the scheduler interface, or the tree-attention kernel API. The system still sees a fixed per-request verification shape; only the token identities inside part of the tree are changed from low-confidence draft candidates to retrieved candidates. This makes the high-concurrency implementation easy to deploy on top of the existing tree worker.

All requests share one GPU-resident adjacency matrix. For online updates, we concatenate the verified tree nodes and target logits from all requests into one flattened tensor and perform a batched top- k successor update. Retrieval is also implemented as GPU indexing from the shared matrix, so it does not introduce CPU-side datastore synchronization or inter-request communication. The current implementation still leaves room for further optimization, such as specialized retrieval kernels and scheduling policies that better exploit cross-request reuse.

E. Runtime Overhead

Table 14 profiles a representative decoding round of Graft on Qwen3-8B. The retrieval branch is built through GPU-resident matrix lookup and overlaps with tree construction. After pruning, merging retrieved nodes into the retained draft tree is almost free (0.015 ms), and rebuilding the tree mask, position IDs, and candidate paths costs 0.570 ms. The dominant latency still comes from target verification (26.275 ms) and the standard per-round KV/input update (8.036 ms), while refreshing the adjacency matrix with target logits costs 0.320 ms. Thus, Graft adds only a small bookkeeping overhead on top of the original draft-then-verify pipeline, and most of the runtime remains in the same verification and cache-update components as tree-based decoding.



Table 14 | **Runtime breakdown of Graft on Qwen3-8B.** Retrieval construction is launched in parallel with tree drafting, so its latency is reported separately and does not add to the critical path. Percentages are normalized by the measured critical-path latency of 43.19 ms.

Component	Draft	Merge	Rebuild	Verify	Matrix Update	Posterior	KV/Input Update
Time (ms)	6.752	0.015	0.570	26.275	0.320	0.221	8.036
Share	15.6%	0.03%	1.3%	62.8%	0.7%	0.5%	18.6%

F. DFlash Demo Implementation

We conduct the DFlash experiment as a lightweight demo on top of the HuggingFace transformers implementation, rather than a full SGLang integration. The goal is to test whether the grafting principle also applies to block drafters, not to provide a fully optimized DFlash deployment. DFlash drafts a block of 16 tokens and then verifies the proposed block with the target model. In this setting, we observe that confidence derived from DFlash draft logits is positively correlated with target acceptance. We therefore use the draft logits as a block-token pruning signal: low-confidence positions are treated as low-utility proposals and can be replaced by retrieved candidates.

Because DFlash follows a chain-style draft-and-verify path rather than a tree-style verifier, we use a simple Graft (TAIL) strategy. After confidence pruning, we keep the reliable DFlash prefix and attach retrieved tokens to the tail of the remaining chain. This design avoids changing the original DFlash pipeline and does not introduce additional tree-verification cost. It also reduces the prefix-dependency risk of retrieval, since retrieved candidates are conditioned on the retained high-confidence prefix instead of being inserted into arbitrary intermediate positions.

Let B be the DFlash block budget. Graft (TAIL) keeps B^{df} DFlash tokens and fills the remaining $B^{\text{ret}} = B - B^{\text{df}}$ slots with retrieved tail tokens. Thus, the total number of verified tokens remains B , matching the original DFlash budget. The target model still verifies every proposed token under the standard speculative acceptance rule, so the procedure remains lossless. In this demo, we reuse the same GPU-resident adjacency matrix as Graft for retrieval, but only attach retrieved tokens at the tail.

This preliminary implementation leaves several directions for future work. First, the DFlash grafting path should be implemented and optimized in high-concurrency settings. Second, more model and task configurations are needed to understand when block-logit confidence is most predictive. Finally, alternative retrieval sources and grafting positions may further improve block drafters, especially if they can preserve the low-latency chain structure while allocating released token budget more effectively.

G. Detailed Related Work

Speculative decoding and draft models. Speculative decoding accelerates autoregressive generation by using a cheap proposal mechanism and then verifying the proposed tokens with the target model (Leviathan et al., 2023; Chen et al., 2023). Early formulations use an independent smaller draft model, while later work improves candidate quality through blockwise prediction, tree verification, and feature-level drafting (Stern et al., 2018; Miao et al., 2024a; Li et al., 2024a,b, 2025). Self-speculative decoding, where the target model reuses its own intermediate layers for drafting and verification, eliminating the need for a separate draft model; this direction was initiated by Draft & Verify and later extended to on-the-fly and dynamically optimized variants such as SWIFT and KNN-SSD (Zhang et al., 2024a; Xia et al., 2025; Song et al., 2026). Medusa-style methods attach auxiliary decoding heads to the target model and generate multiple future-token proposals in parallel (Cai et al., 2024). EAGLE-style methods instead reuse target hidden features and construct a draft tree, offering a strong practical baseline for lossless LLM acceleration (Li et al., 2024a,b, 2025). Graft is instantiated on this verification path in our main experiments, but the mechanism changes how a fixed-budget candidate tree is populated: instead of spending all slots on parametric drafting, it releases low-confidence branches and fills the budget with retrieved candidates.

Dynamic tree construction and adaptive speculation. Tree-based speculative decoding increases the chance of accepting multiple tokens by verifying multiple candidate paths in one target forward (Miao et al., 2024a). Static tree methods are easy to deploy but may waste budget on low-confidence branches. Adaptive speculation

methods adjust the draft length or tree topology based on confidence, acceptance history, or token probabilities (Mamou et al., 2024; Zhang et al., 2024b; Brown et al., 2024; Hu et al., 2026). These methods expose a central trade-off: pruning or early stopping can reduce draft-side overhead, but it may also remove valid continuations and lower MAT. ECHO further studies this issue in high-concurrency settings and formulates elastic budget scheduling across requests (Hu et al., 2026). Graft follows the same broad motivation of reducing wasted candidates, but it avoids cross-request budget reallocation in the deployed high-concurrency version. The released slots are reused by retrieval grafting under the same per-request tree size, which preserves the original tree-attention shape.

Retrieval-based speculative decoding. Retrieval is an attractive proposal source because repeated local patterns can be reused with little draft-model computation. Lookahead decoding, PLD, REST, Token Recycling, LogitSpec, SAMD, and Ouroboros explore different retrieval or matching mechanisms for candidate generation (Fu et al., 2024; Saxena, 2023; He et al., 2024; Luo et al., 2024; Liu et al., 2025b; Hu et al., 2025; Zhao et al., 2024a). However, retrieval-only methods often depend on prompt-local overlap, CPU-side structures, suffix automata, or phrase-level matching, which can limit their speedup and make integration with high-throughput tree verification nontrivial. Graft treats retrieval as a complementary branch rather than a replacement for the parametric drafter. The GPU-resident adjacency matrix lets retrieval run as matrix indexing, and the retrieved nodes are grafted into the same hybrid tree verified by the target model.

Parallel and system-level speculative decoding. A separate line of work studies how to overlap or pipeline drafting and verification. Parallel speculative decoding reduces mutual waiting between the draft and target model by adapting draft length or running draft and verification work concurrently (Liu et al., 2024; Shen et al., 2025, 2026). Lookahead-style frameworks also try to break strict next-token dependency by constructing multiple candidate branches without an external draft model (Fu et al., 2024; Zhao et al., 2024b). These methods emphasize pipeline utilization and rollback reduction, while high-concurrency systems focus on the interaction between speculation and batched execution. Graft is orthogonal to these directions: it does not introduce a new pipeline schedule, but makes the tree itself more useful under a fixed verification budget. In SGLang, this allows a simple implementation that keeps the per-request candidate shape unchanged and avoids modifying CUDA graph or tree-attention interfaces.

Speculative decoding for long context. Long-context generation shifts the bottleneck toward KV-cache traffic and attention memory bandwidth. LongSpec improves long-context lossless speculative decoding through efficient drafting and verification (Yang et al., 2025b), while SpecPV studies partial verification for long-context self-speculative decoding (Tan et al., 2025). Other long-context systems explore sparse KV, partial KV, hierarchical speculation, or cache compression to reduce memory pressure. Graft is motivated by a different but compatible observation: longer prompts expose more local transition evidence, making retrieval more useful, while tree drafting over long contexts becomes increasingly expensive. By combining a long-context tree drafter with GPU-resident retrieval grafting, Graft improves both short-context and long-context speculative decoding without changing the target verification rule.

Block and diffusion drafters. Recent work also studies non-autoregressive or block-level drafters that reduce the serial cost of proposal generation. KVShot provides the first systematic study of long-range decay in hidden-state-based speculative drafters, explores KV-cache reuse to improve long-horizon acceptance, and suggests block-wise paradigms as a promising direction (Liu et al., 2026b). DFlash uses a block diffusion drafter to generate an entire token block in parallel and achieves strong speedups over autoregressive tree drafting on several tasks (Chen et al., 2026). Nevertheless, block proposals can still suffer from mismatch with target-model autoregressive verification, especially on harder chat or instruction-following data. Retrieval grafting is a natural complement in this setting: confidence can be used to prune unreliable block tokens, and retrieved continuations can fill the released budget with candidates supported by local history. Our preliminary DFlash results in Sec. 4 suggest that this direction is promising.

Multimodal speculative decoding. Speculative decoding for multimodal and vision-language models introduces additional challenges beyond text-only LLMs. The visual prefix can be long, heterogeneous, and expensive to encode, while the acceptance behavior depends on both visual alignment and text continuation quality. Recent surveys and systems identify efficient inference for large vision-language models as an emerging bottleneck (Zhang et al., 2026). SpecVLM (Ji et al., 2025) is the first to explore training-free speculative decoding for Video-LLMs through vision-aware token pruning on the draft side. LVSpec (Ji et al., 2026) further



extends this line of work by introducing vision-aware loose verification for Video-LLMs. ParallelVLM extends lossless acceleration to video-LLMs by considering visual-alignment-aware parallel speculative decoding (Kong et al., 2026). These works suggest that future speculative decoding methods must account for modality-specific proposal quality, visual-token compression, and cross-modal cache reuse. Graft is currently evaluated on text generation, but its grafting principle is compatible with multimodal speculation: retrieved candidates can be treated as additional text proposals, while target verification remains responsible for preserving the output distribution.